GIÁO TRÌNH ĐÀO TẠO KỸ SƯ FRONTEND CHUYÊN SÂU

GIÁO TRÌNH CSS3

THIẾT KẾ GIAO DIỆN WEB

Từ Nền Tảng Layout đến Flexbox, Grid, CSS Variables & Animations

■ CSS Foundations • ■ Flexbox & Grid • ■ Responsive • ■ BEM & Variables

TÀI LIỆU CHUYÊN SÂU TẬP TRUNG CSS3

Hệ thống hóa toàn diện kiến thức CSS hiện đại • Bố cục Flexbox & Grid • Minh họa trực quan chuẩn UI/UX

ĐỘI NGŨ BIÊN SOẠN WEB

Tài liệu lưu hành nội bộ

NĂM 2026

Phiên bản Standalone

ĐỘI NGŨ BIÊN SOẠN WEB

TÀI LIỆU LƯU HÀNH NỘI BỘ – CHUYÊN ĐỀ CSS3

GIÁO TRÌNH CSS3

THIẾT KẾ GIAO DIỆN WEB

Từ Nền Tảng Layout đến Flexbox, Grid, CSS Variables & Animations

---

TẬP TRUNG BIÊN SOẠN & THỰC HÀNH CSS3

Biên soạn chi tiết • Minh họa trực quan • Tối ưu tốc độ biên dịch

NĂM 2026

Phiên bản XeLaTeX Unicode

MỤC LỤC

Giáo Trình Chuyên Sâu CSS3 – Thiết Kế Giao Diện & Bố Cục Trang Web

PHẦN I

CSS3 – Thiết Kế Giao Diện & Bố Cục Trang Web

CHƯƠNG 1

Nền Tảng CSS: Cascading Style Sheets, Box Model & Thuộc Tính

Outline

1.1 Sơ Lược & Khái Niệm Về CSS

CSS (**Cascading Style Sheets**) giúp bạn thiết kế và định dạng giao diện trang web. Nó quyết định mọi thứ về hình thức hiển thị, từ màu sắc, kích thước chữ đến bố cục và hiệu ứng.

[Khái Niệm] Khái Niệm CSS Là Gì?

CSS (Cascading Style Sheets) là ngôn ngữ dùng để mô tả cách trình bày của một tài liệu HTML hoặc XML. Nó giúp định dạng màu sắc, kiểu chữ, bố cục và hiệu ứng cho trang web. CSS giúp **tách biệt phần nội dung (HTML) với phần hiển thị**, giúp trang web dễ quản lý, tăng tính bảo trì và tối ưu hiệu suất tải trang.

1.1.1 Mục Đích Cốt Lõi Của Ngôn Ngữ CSS

1. **Tách biệt nội dung và trình bày**: Thẻ HTML tập trung vào nội dung ngữ nghĩa, CSS lo phần thiết kế giao diện thị giác.
2. **Giúp trang web đẹp và chuyên nghiệp**: Mang lại giao diện hiện đại, nâng cao trải nghiệm thị giác.
3. **Dễ bảo trì và tái sử dụng**: Một tập tin CSS duy nhất có thể dùng và định kiểu đồng bộ cho nhiều trang web.
4. **Cải thiện trải nghiệm người dùng và tối ưu tốc độ tải trang**: Giảm kích thước file HTML, tối ưu hóa bộ nhớ đệm trình duyệt (**browser cache**).

1.2 Cú Pháp CSS (CSS Syntax)

1.2.1 Cấu Trúc Cơ Bản Của Cú Pháp CSS

Một quy tắc CSS bao gồm hai thành phần chính:

- **Bộ chọn (Selector)** – Xác định phần tử HTML cần áp dụng CSS.
- **Khối khai báo (Declaration Block)** – Chứa các thuộc tính và giá trị đặt trong cặp dấu ngoặc nhọn `{}`.

Cú Pháp Định Dạng CSS Chuẩn

```
selector {
    thuoc_tinh: gia_tri;
    thuoc_tinh: gia_tri;
}
```

BROWSER PREVIEW — **Kết Quả: Sơ Đồ Cấu Trúc Khai Báo CSS**



Trình duyệt quét cây DOM để tìm tất cả thẻ khớp `h1` và áp dụng đồng thời các cặp `thuộc tính: giá trị` bên trong cặp ngoặc nhọn.

1.2.2 Ví Dụ Cụ Thể & Giải Thích Cú Pháp

Khai Báo Style Cho Thẻ h1

```
h1 {
    color: red;
    font-size: 24px;
}
```

BROWSER PREVIEW — **Kết Quả Hiển Thị: Thẻ h1 Chữ Đỏ, Cỡ Chữ 24px**

<!-- Cửa sổ hiển thị trang web -->

Tiêu Đề Văn Bản h1

Đây là đoạn văn bản nội dung đi kèm để đối chứng tỷ lệ với tiêu đề h1 màu đỏ cỡ chữ 24px.

`h1 { color: red; font-size: 24px; } → Rendered CSS`

[Ghi Chú] Giải Thích Chi Tiết Cú Pháp

- `h1` → Chọn tất cả thẻ `<h1>` trên trang.
- `color: red;` → Thiết lập màu chữ đỏ.
- `font-size: 24px;` → Thiết lập cỡ chữ `24px`.

1.3 Các Cách Thêm Style Vào Trang Web (inline, internal, external)

CSS có thể được áp dụng vào HTML theo 3 cách:

[Bảng] Bảng So Sánh 3 Cách Thêm Style Vào Trang Web

Cách viết	Mô tả	Ưu điểm	Nhược điểm
Inline CSS	Viết trực tiếp vào thẻ HTML bằng thuộc tính `style`.	Dễ dùng, nhanh chóng.	Khó bảo trì, không tái sử dụng được.
Internal CSS	Viết trong thẻ `<style>` trong file HTML.	Dễ chỉnh sửa toàn bộ trang, không cần file ngoài.	Không tách biệt nội dung và kiểu dáng.
External CSS	Link tới file `.css` bên ngoài qua thẻ `<link>`.	Tái sử dụng cao, dễ bảo trì.	Phải tải thêm file, trang load lâu hơn chút.

1.3.1 Inline CSS (Khai Báo Trong Thuộc Tính style)

Viết trực tiếp vào thẻ HTML bằng thuộc tính `style`.

Mã Nguồn Inline CSS

```
<h1 style="color: red; font-size: 30px;">Header Title</h1>
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Inline CSS Áp Dụng Trực Tiếp Trong Thẻ HTML**

Header Title

```
<h1 style="color: red; font-size: 30px;">
```

(→ Thuộc tính CSS nằm trực tiếp trên thẻ HTML, có độ ưu tiên cao nhưng gây khó khăn khi tái sử dụng).

→ **Hạn chế**: **Không khuyến khích** vì làm code khó bảo trì.

1.3.2 Internal CSS (Khai Báo Trong Thẻ `<style>`)

Viết trong thẻ `<style>` trong file HTML (bên trong `<head>`). Dùng khi chỉ áp dụng CSS cho một trang web duy nhất.

Mã Nguồn Internal CSS

```
<head>
  <style>
    h1 {
      color: blue;
      font-size: 24px;
    }
  </style>
</head>
```

BROWSER PREVIEW **Kết Quả Hiển Thị: Internal CSS Được Định Kiểu Từ Thẻ <style>**

Tiêu Đề Xanh Dương

```
<style> h1 { color: blue; font-size: 24px; } </style>
```

(→ Cung cấp quy tắc định kiểu tập trung cho toàn bộ trang web trong cùng một file HTML).

1.3.3 External CSS (Liên Kết File Ngoại Qua `<link>`)

Viết CSS trong file `.css` riêng biệt, sau đó liên kết vào HTML bằng thẻ `<link>`.

Liên Kết File CSS Qua Thẻ Link

```
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

Nội Dung Trong File style.css

```
/* Sample Text */
p {
    color: green;
    font-size: 16px;
}
```

BROWSER PREVIEW Kết Quả Hiển Thị: External CSS Liên Kết Từ File style.css

Đoạn văn bản màu xanh lá được định kiểu từ file `style.css` bên ngoài.

`<link rel="stylesheet" href="style.css"> → p { color: green; font-size: 16px; }`

(→ Cách chuẩn mực chuyên nghiệp: Tách biệt triệt để nội dung HTML và kiểu dáng CSS).

→ **Lời khuyên**: **Khuyến khích sử dụng** vì giúp code gọn gàng và dễ bảo trì.

1.4 Một Số Thuộc Tính CSS Cơ Bản Phổ Biến

Để bắt đầu định kiểu giao diện trang web, lập trình viên cần nắm vững các thuộc tính CSS cơ bản và có tần suất xuất hiện cao nhất trong hầu hết mọi phần tử giao diện. Bảng tra cứu dưới đây tổng hợp các thuộc tính nền tảng, chức năng cốt lõi cùng cú pháp minh họa trực quan.

1.4.1 Bảng Tra Cứu Nhanh Các Thuộc Tính CSS Cơ Bản

[Bảng] Bảng Tra Cứu Các Thuộc Tính CSS Cơ Bản Phổ Biến

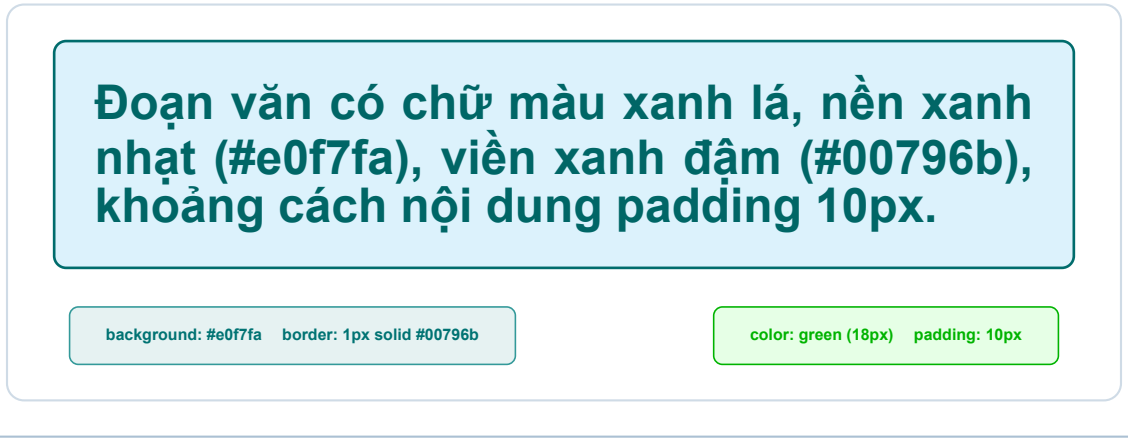
Thuộc tính	Chức năng	Ví dụ
`color`	Đặt màu chữ.	`color: red;`
`font-size`	Đặt kích thước chữ.	`font-size: 20px;`
`background`	Đặt màu nền hoặc ảnh nền.	`background: #f0f0f0;`
`border`	Tạo viền cho phần tử.	`border: 2px solid black;`
`padding`	Khoảng cách bên trong phần tử.	`padding: 10px;`
`margin`	Khoảng cách bên ngoài phần tử.	`margin: 20px;`
`text-align`	Căn chỉnh văn bản.	`text-align: center;`
`display`	Quyết định kiểu hiển thị của phần tử.	`display: flex;`

Ví Dụ Kết Hợp Thuộc Tính Định Kiểu Cho Thẻ p:

Ví Dụ Định Kiểu Thẻ p Tổng Hợp

```
p {
    color: green;
    font-size: 18px;
    background: #e0f7fa;
    padding: 10px;
    border: 1px solid #00796b;
}
```

BROWSER PREVIEW **Kết Quả Hiển Thị Thẻ p: Tổng Hợp Màu Chữ, Nền, Viền & Padding**



1.5 Chuyên Đề Hệ Thống Đơn Vị Đo Trong CSS (CSS Units Masterclass)

[LỘ TRÌNH CHUYÊN ĐỀ 1.5] BẢN ĐỒ KIẾN THỨC HỆ THỐNG ĐƠN VỊ ĐO CSS

Nhánh A: Phân Loại & Đơn Vị Tuyệt Đối

- **1.5.1:** Tổng quan & Phân loại Absolute vs Relative
- **1.5.2:** Đơn vị tuyệt đối `px` Masterclass
- **1.5.3:** Đơn vị phần trăm `%` & Bài toán `height`

Nhánh B: Đơn Vị Tương Đối & Responsive UI

- **1.5.4:** Đơn vị tương đối `em` & Cạm bẫy nhân dồn
- **1.5.5:** Đơn vị gốc `rem` & Kỹ thuật Root Reset 62.5%
- **1.5.6 – 1.5.8:** Viewport (`vw`, `vh`, `dvh`, `svh`) & `clamp()`

1.5.1 Tổng Quan & Phân Loại Các Đơn Vị Trong CSS (CSS Units Classification)

Trong CSS, các đơn vị đo (**CSS Units**) quyết định kích thước của văn bản, khoảng cách cách lề (`margin`, `padding`), chiều rộng (`width`), chiều cao (`height`) và toàn bộ bố cục của trang web. Việc lựa chọn đúng loại đơn vị đo là yếu tố cốt lõi giúp giao diện web hiển thị chuẩn xác, đáp ứng linh hoạt trên mọi kích thước màn hình (**Responsive Web Design**).

CSS Units được chia làm 2 nhóm chính: **Đơn vị Tuyệt đối (Absolute Units)** và **Đơn vị Tương đối (Relative Units)**.

1. Nhóm Đơn Vị Tuyệt Đối (Absolute Units)

Đơn vị tuyệt đối là các đơn vị cố định, có kích thước không bao giờ thay đổi bất kể kích thước màn hình, độ phân giải thiết bị hay bối cảnh phần tử cha.

[Bảng] Bảng Chi Tiết Các Đơn Vị Tuyệt Đối (Absolute Units)

Đơn Vị	Ý Nghĩa	Ghi Chú & Đặc Điểm
`px`	Pixels	Phổ biến nhất, kích thước cố định trên màn hình digital.
`pt`	Point ($1\text{pt} = 1/72$ inch)	Dành cho in ấn (ít dùng trên giao diện web).
`cm`	Centimeter	Dùng cho thiết kế in ấn, không phù hợp màn hình.
`mm`	Millimeter	Tương tự cm, dùng cho in ấn.
`in`	Inch	$1\text{in} = 2.54\text{cm} = 96\text{px}$.
`pc`	Pica ($1\text{pc} = 12\text{pt}$)	Hiếm dùng trên web.

1.5.2 Chuyên Đề Đơn Vị Pixel (px Masterclass Trong CSS)

Trong CSS, `px` (viết tắt của **Pixel**) là đơn vị độ dài tuyệt đối vô cùng phổ biến và là một trong những đơn vị cơ bản nhất được sử dụng để xác định kích thước cho các thuộc tính như: `width`, `height`, `margin`,

`padding`, `font-size`, `border-width`, `border-radius`, `box-shadow`, `text-shadow`, `min-width`, `max-width`, `min-height`, `max-height`...

1. Pixel (px) Trong Ngữ Cảnh Màn Hình Là Gì?

[Khái Niệm] Khái Niệm Pixel Và Pixel Tham Chiếu Trong CSS

- **Pixel trên màn hình (Picture Element)**: Trong thế giới kỹ thuật số, một pixel (viết tắt của **Picture Element**) là điểm ảnh nhỏ nhất trên màn hình phần cứng có thể điều khiển riêng lẻ để hiển thị một màu sắc. Màn hình của bạn được tạo thành từ hàng triệu pixel xếp cạnh nhau.
- **Pixel tham chiếu trong CSS (`px`)**: Khi bạn sử dụng đơn vị `px` trong CSS, bạn đang tham chiếu đến một **Pixel tham chiếu (Reference Pixel)**. Khác với pixel vật lý, pixel tham chiếu trong CSS là một đơn vị tuyệt đối – giá trị của nó cố định và không thay đổi dựa trên kích thước phần tử cha hay kích thước trình duyệt (**Viewport**).
- **Tính cố định tuyệt đối**: Nếu bạn đặt `width: 200px;` cho một phần tử, nó sẽ luôn có chiều rộng đúng 200 pixel tham chiếu. Giá trị này không bị ảnh hưởng bất kể phần tử nằm trong một thẻ `<div>` nhỏ hay một `<div>` lớn hơn.

2. Sự Khác Biệt Giữa Pixel CSS Và Pixel Vật Lý (HiDPI / Retina Display):

Đây là điểm quan trọng và thường gây nhầm lẫn nhất trong thiết kế giao diện web:

- **Pixel vật lý (Physical Pixel)**: Là chấm sáng nhỏ nhất trên màn hình phần cứng của thiết bị.
- **Pixel CSS (CSS Reference Pixel)**: Là đơn vị trừu tượng mà trình duyệt sử dụng để tính toán kích thước.
- **Cách trình duyệt xử lý trên các màn hình**:
  - **Màn hình tiêu chuẩn (Standard DPI)**: 1px CSS thường tương đương với 1 pixel vật lý.
  - **Màn hình mật độ cao (HiDPI / Retina)**: Để nội dung hiển thị sắc nét hơn và không bị "rỗ", trình duyệt sẽ tự động "phóng to" pixel CSS. Trên màn hình Retina, 1px CSS có thể được hiển thị bằng 2×2 (4) hoặc thậm chí 3×3 (9) pixel vật lý.
- **Ví dụ thực tế**: Một đoạn văn bản có `font-size: 16px;` sẽ có kích thước vật lý tương tự nhau trên cả màn hình tiêu chuẩn và màn hình Retina. Tuy nhiên, trên màn hình Retina, nó được vẽ bằng nhiều pixel vật lý hơn, khiến chữ hiển thị mịn màng và sắc nét hơn.
- **Mục đích cốt lõi**: Đảm bảo các phần tử trên trang web có kích thước ổn định và dễ đọc về mặt vật lý trên các thiết bị khác nhau, bất kể mật độ điểm ảnh.

3. Đơn Vị Tuyệt Đối Là Gì?

Đơn vị `px` là một đơn vị tuyệt đối, có nghĩa là giá trị của nó cố định và không thay đổi dựa trên kích thước của phần tử cha hoặc kích thước khung nhìn (**Viewport**). Nếu đặt `width: 200px;` cho một thẻ `<div>`, thẻ đó sẽ luôn có chiều rộng đúng 200 pixel tham chiếu, bất kể kích thước màn hình hay kích thước phần tử chứa nó.

4. Ví Dụ Đơn Giản Khai Báo px Trong CSS:

Khai Báo Kích Thước Khối .box Bằng Đơn Vị px

```
<style>
.box {
  width: 200px;
  height: 100px;
  font-size: 16px;
  padding: 10px;
  border: 2px solid black;
}
</style>

<div class="box">Khoi Box dung px</div>
```

BROWSER PREVIEW — **Mô Phỏng Trực Quan Khối .box Đơn Vị px**



(→ Khung viền đen dày chuẩn `2px solid black`, khoảng đệm padding `10px` cách mép viền, kích thước cố định tuyệt đối).

5. Đặc Điểm Của Đơn Vị px (Ưu Điểm & Nhược Điểm):

[Bảng] Bảng Tổng Hợp Ưu Điểm Và Nhược Điểm Của Đơn Vị px

Ưu Điểm Của Pixel (px)	Nhược Điểm & Cân Nhắc
Dễ kiểm soát kích thước chính xác tuyệt đối.	Không linh hoạt với màn hình quá lớn hoặc quá nhỏ.
Rất trực quan, dễ hình dung con số đại diện.	Khó tạo Responsive Web Design (cần viết nhiều Media Queries).
Phù hợp khi thiết kế chuẩn từng Pixel (**Pixel-Perfect**).	Không tự động co giãn theo thiết bị hoặc cài đặt Zoom phông chữ của người dùng.
Nhất quán trên mọi trình duyệt và hệ điều hành.	Gây cản trở khả năng truy cập (**Accessibility**) nếu dùng cho `font-size` chính.
Tuyệt vời cho phần tử cố định (logo, icon, đường viền).	Có thể bị mờ hình ảnh raster nếu không dùng ảnh HiDPI/$2\times$.

6. So Sánh px Với Các Đơn Vị Khác (em, rem, %, vw, vh):

[Bảng] Bảng So Sánh Yếu Tố Phụ Thuộc Của Các Đơn Vị CSS

Đơn Vị	Ý Nghĩa	Phụ Thuộc Vào Ai?
`px`	Pixel tuyệt đối	Không phụ thuộc ai (Cố định tuyệt đối)
`em`	Tương đối theo phông chữ gần nhất	Dựa vào `font-size` của phần tử cha
`rem`	Tương đối theo phông chữ gốc	Dựa vào `font-size` của thẻ `<html>`
`%`	Tỷ lệ phần trăm	Dựa vào kích thước của phần tử cha
`vw / vh`	Phần trăm khung hình	Dựa vào kích thước màn hình (**Viewport**)

[Bảng] Bảng Ma Trận Sự Khác Biệt Tính Chất Giữa px Và Các Đơn Vị Tương Đối

Thuộc Tính	px	em	rem	%	vw / vh
`Cố định`	[Có]	[Không]	[Không]	[Không]	[Không]
`Responsive`	[Không]	[Có] (tùy cha)	[Có] (toàn trang)	[Có] (tùy cha)	[Có] (theo Viewport)
`Dễ kiểm soát`	[Có] (nhỏ lẻ)	[Khó] (dễ bị lồng)	[Có] (tùy ngữ cảnh)	[Có] (layout lớn)	[Có] (layout tràn màn)

7. Các Trường Hợp Sử Dụng Phổ Biến Của Pixel (px):

[Bảng] Bảng Tổng Hợp 6 Trường Hợp Sử Dụng Phổ Biến Của Đơn Vị Pixel

Trường Hợp Sử Dụng	Thuộc Tính CSS	Giải Thích & Đánh Giá
`Độ dày đường viền`	`border-width`	Thích hợp nhất cho viền cố định 1px – 3px, không bị biến dạng.
`Khoảng cách lề cố định`	`padding, margin`	Giữ khoảng cách nhỏ chuẩn xác (ví dụ 10px, 15px, 20px).
`Kích thước Icon / Logo`	`width, height`	Đảm bảo biểu tượng nhỏ không bị lệch tỷ lệ hiển thị.
`Giới hạn kích thước tối đa`	`max-width, min-width`	Giới hạn chiều rộng khung nội dung (ví dụ 960px, 1200px).
`Hiệu ứng viền & bóng đổ`	`border-radius, box-shadow`	Tạo độ bo góc và độ nhòe bóng đổ chuẩn xác từng pixel.
`Cỡ chữ cố định (Font Size)`	`font-size`	Kiểm soát cỡ chữ chuẩn tuyệt đối (nhược điểm: khó zoom riêng chữ).

Mã Nguồn Mẫu Tổng Hợp Các Trường Hợp Sử Dụng px Thực Tế

```css
/* 1. Do day duong vien co dinh: */
.element {
  border: 1px solid #ccc;
}

/* 2. Khoang cach padding va margin nho: */
.button {
  padding: 10px 20px;
  margin-top: 15px;
}

/* 3. Kich thuoc Icon va Logo nho: */
.icon {
  width: 24px;
  height: 24px;
}

/* 4. Gioi han chieu rong khung noi dung: */
.content {
  max-width: 960px; /* Noi dung khong bao gio rong hon 960px */
}

/* 5. Bo goc va tao bong do: */
.card {
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

/* 6. Co chu co dinh: */
h1 {
  font-size: 32px;
}
```

BROWSER PREVIEW Mô Phỏng Trực Quan Kết Quả Hiển Thị Các Trường Hợp Sử Dụng px



8. Khi Nào Nên Dùng px Và Khi Nào Nên Dùng Đơn Vị Tương Đối?

- **Nên sử dụng `px` khi**:
  1. Cần độ chính xác cao tuyệt đối và kích thước cố định (đường viền, icon, khoảng cách nhỏ cụ thể).
  2. Đảm bảo một phần tử không thay đổi kích thước bất kể kích thước màn hình hay phần tử cha.
  3. Đối với các thuộc tính hiệu ứng kỹ thuật như `border-radius`, `box-shadow`, `text-shadow`.
  4. Làm thiết kế tĩnh (**Fixed Layout**) hoặc UI nội bộ không yêu cầu responsive quá linh hoạt.
- **Nên sử dụng đơn vị tương đối (%, `em`, `rem`, `vw`, `vh`) khi**:
  1. Xây dựng bố cục đáp ứng (**Responsive Layout**) tự động co giãn theo kích thước màn hình.
  2. Đảm bảo khả năng truy cập (**Accessibility**), đặc biệt là cho `font-size` toàn trang.
  3. Duy trì tỷ lệ tương quan giữa các phần tử với nhau hoặc với khung hình Viewport.

[Ghi Chú] Kết Luận Thực Tế Về Sử Dụng Đơn Vị Pixel

Trong thực tế phát triển web hiện đại, một trang web chuyên nghiệp **luôn kết hợp cả đơn vị tuyệt đối (`px`) và đơn vị tương đối (`rem`, `em`, `%`)** để đạt được sự cân bằng hoàn hảo giữa kiểm soát chính xác và tính linh hoạt đáp ứng.

[Cảnh Báo] Lưu Ý Về Đơn Vị Tuyệt Đối (Absolute Units)

Các đơn vị tuyệt đối (**Absolute Units**) **không phản ứng với kích thước màn hình hoặc cài đặt phông chữ của trình duyệt**, do đó chúng **không có tính responsive**. Hạn chế dùng `px` cho bố cục lớn hoặc kích thước phông chữ toàn trang.

[Bảng] Bảng Chi Tiết Các Đơn Vị Tương Đối (Relative Units)

Đơn Vị	Ý Nghĩa	Dựa Vào Đâu / Ghi Chú
`%`	Tỷ lệ phần trăm	So với phần tử cha (thường là `width`).
`em`	Relative to `font-size` của chính nó	Kế thừa từ phần tử cha nếu không tự thiết lập.
`rem`	Relative to `font-size` của `<html>`	Dễ kiểm soát hơn `em`, làm chuẩn cho toàn trang.
`vw`	Viewport width – theo chiều ngang	$1\text{vw} = 1\%$ chiều rộng cửa sổ trình duyệt.
`vh`	Viewport height – theo chiều cao	$1\text{vh} = 1\%$ chiều cao cửa sổ trình duyệt.
`vmin`	Nhỏ hơn giữa `vw` và `vh`	Phản ứng linh hoạt theo chiều nhỏ nhất của màn hình.
`vmax`	Lớn hơn giữa `vw` và `vh`	Hiếm dùng, theo chiều lớn nhất của màn hình.
`ch`	Chiều rộng ký tự số "0" của phông hiện tại	Thường dùng để giới hạn độ rộng cho vùng nhập văn bản (`input text`).
`ex`	Chiều cao ký tự "x" của phông hiện tại	Ít dùng, không chính xác trên mọi phông chữ.
`lh`	Line height hiện tại	Tương đối với thuộc tính `line-height` của phần tử.

1.5.3 Chuyên Đề Đơn Vị Phần Trăm (% Masterclass Trong CSS)

Đơn vị phần trăm (%) trong CSS là một đơn vị tương đối cực kỳ mạnh mẽ và linh hoạt, thường được sử dụng làm nòng cốt để tạo ra các bố cục trang web tự động co giãn và đáp ứng (**Responsive Web Design**).

1. Đơn Vị Tương Đối (%) Là Gì?

[Khái Niệm] Khái Niệm Đơn Vị Tương Đối Phần Trăm (%)

Trước hết, cần hiểu % là một **đơn vị tương đối** (**Relative Unit**). Điều này có nghĩa là giá trị thực tế của nó không cố định mà sẽ thay đổi tùy thuộc vào một **yếu tố tham chiếu** khác. Yếu tố tham chiếu này thường là kích thước của phần tử cha (**Parent Element**) hoặc một thuộc tính cụ thể của chính phần tử đó.

Đơn vị % được áp dụng rộng rãi cho nhiều thuộc tính CSS quan trọng như: `width`, `height`, `margin`, `padding`, `font-size`, `line-height`, `transform`, `background-size`, `top`, `left`, `right`, `bottom`...

2. Nguyên Tắc Hoạt Động Chung & Công Thức Tính Đơn Vị %:

Khi bạn gán một giá trị % cho một thuộc tính CSS, trình duyệt sẽ tính toán giá trị độ dài thực tế (**Computed Value** tính bằng pixel) bằng cách lấy tỷ lệ phần trăm đó nhân với giá trị cơ sở (**Base Value**):

[Khái Niệm] Công Thức Chuyển Đổi Đơn Vị Phần Trăm Sang Pixel Thực Tế

$$\text{Giá Trị Thực Tế (px)} = \left(\frac{\text{Giá Trị Phần Trăm}}{100}\right) \times \text{Giá Trị Cơ Sở (px)}$$

- **Giá trị phần trăm**: Là con số bạn chỉ định trong CSS (ví dụ: `50%`, `20%`, `150%`...).
- **Giá trị cơ sở**: Là giá trị tham chiếu mà phần trăm dựa vào (ví dụ: `width` của phần tử cha, `font-size` của phần tử cha...).
- **Giá trị thực tế**: Là số pixel thực tế được trình duyệt tính ra và áp dụng lên màn hình.

[Bảng] Bảng Minh Họa Các Phép Tính Chuyển Đổi % Cơ Bản Trong CSS

Thuộc Tính	Khai Báo CSS	Công Thức Tính Toán	Giá Trị Thực Tế
`width`	Cha 500px, con `width: 60%`	`(60 / 100) * 500px`	300px
`font-size`	`body` 20px, `p` `font-size: 150%`	`(150 / 100) * 20px`	30px
`padding`	Khung 400px, `padding: 10%`	`(10 / 100) * 400px`	40px (cả 4 phía)

[Bảng] Bảng Tóm Tắt Nguyên Lý Hoạt Động Của Đơn Vị %

Thành Phần	Ý Nghĩa / Quy Tắc
Bản chất đơn vị	Đơn vị tương đối (**Relative Unit**).
Điểm tựa tính toán	Phụ thuộc vào thuộc tính tương ứng của phần tử cha hoặc chính nó.
Công thức cốt lõi	Giá trị thực tế $=$ (Phần trăm$/100$) $\times$ Giá trị cơ sở.

3. Điểm Tham Chiếu Của Đơn Vị % Theo Từng Thuộc Tính CSS:

Đây là điểm quan trọng nhất cần nắm rõ khi sử dụng %, vì điểm tham chiếu thay đổi tùy thuộc vào thuộc tính mà bạn áp dụng nó:

[Bảng] Bảng Tra Cứu Điểm Tham Chiếu Của Đơn Vị % Theo Thuộc Tính CSS

Thuộc Tính CSS	Phụ Thuộc Vào...	Giải Thích Chi Tiết & Điểm Tham Chiếu
`width & height`	`width` hoặc `height` của phần tử cha	Phần tử con có kích thước bằng tỷ lệ phần trăm của phần tử cha.
`padding & margin`	`width` của phần tử cha	**Lưu ý cốt lõi**: Cả 4 hướng (`top`, `bottom`, `left`, `right`) đều tính theo `width` phần tử cha!
`font-size`	`font-size` của phần tử cha	Kích thước chữ con tính theo tỷ lệ phần trăm cỡ chữ phần tử cha gần nhất.
`line-height`	`font-size` của chính phần tử đó	Tính dựa trên kích thước phông chữ thực tế của phần tử hiện tại.
`transform`	Kích thước của chính phần tử đó	Tính dựa trên chiều rộng hoặc chiều cao của chính phần tử đang biến đổi.
`background-size`	Khung chứa ảnh nền phần tử	Tính dựa trên kích thước vùng bao chứa ảnh nền.

a) Với width (chiều rộng) và height (chiều cao):

- **Điểm tham chiếu**: Giá trị `width` hoặc `height` của phần tử cha (**Parent Element**).
- **Cách hoạt động**: Phần tử con sẽ có chiều rộng/chiều cao bằng một tỷ lệ phần trăm nhất định của phần tử cha.

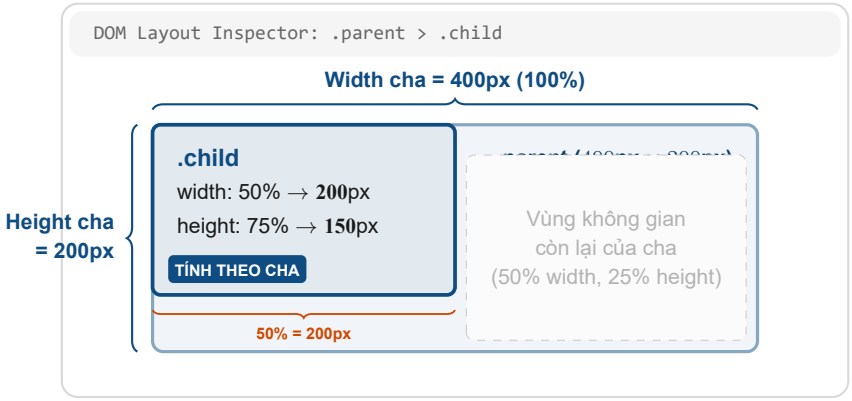
Ví Dụ Tính width và height Của Khối Con Theo % Khối Cha

```
<div class="parent" style="width: 400px; height: 200px; background: #e0e0e0;">
    <div class="child" style="width: 50%; height: 75%; background: #00bcd4; color
    : white;">
        Child Block
    </div>
</div>
```

[Khái Niệm] Giải Thích Chi Tiết Phép Tính Kích Thước Khối Con

- **Chiều rộng thực tế**: `width: 50%` $\rightarrow (50/100) \times 400\text{px} = \mathbf{200}\text{px}$.
- **Chiều cao thực tế**: `height: 75%` $\rightarrow (75/100) \times 200\text{px} = \mathbf{150}\text{px}$.

BROWSER PREVIEW **Mô Phỏng Trực Quan Kết Quả Hiển Thị Của Khối .child (%)**



(→ Kết quả: Kích thước .child được tính tự động dựa trên độ dài 400px × 200px của .parent).

b) Với padding (khoảng đệm) và margin (khoảng lề):

- **Điểm tham chiếu**: Giá trị `width` **(chiều rộng) của phần tử cha**.
- **Cách hoạt động (Điểm thường gây nhầm lẫn nhất)**:
  - Cả `padding-top`, `padding-bottom`, `margin-top`, `margin-bottom` **(chiều dọc) đều được tính dựa trên chiều rộng của phần tử cha**, chứ KHÔNG phải chiều cao.
  - `padding-left`, `padding-right`, `margin-left`, `margin-right` **(chiều ngang)** cũng dựa trên chiều rộng của phần tử cha.
- **Lý do thiết kế**: Nhằm duy trì tỷ lệ khung hình (**Aspect Ratio**) và tính linh hoạt trong bố cục, đặc biệt là khi tạo các khối nội dung co giãn có tỷ lệ cố định.

Mã Nguồn Minh Họa padding và margin Theo Chiều Rộng Phần Tử Cha

```
<div class="parent" style="width: 500px;">
    <div class="child" style="padding-top: 10%; margin-left: 5%; background: #fff3e0; border: 1px solid #ff9800;">
        Noi dung khoi con
    </div>
</div>
```

[Khái Niệm] Giải Thích Phép Tính padding & margin Theo %

Trong ví dụ trên với phần tử cha có `width: 500px`:

- `padding-top: 10%`: Khoảng đệm trên thực tế là $(10/100) \times 500\text{px} = \mathbf{50}\text{px}$.
- `margin-left: 5%`: Khoảng lề trái thực tế là $(5/100) \times 500\text{px} = \mathbf{25}\text{px}$.

BROWSER PREVIEW **Mô Phỏng Trực Quan padding-top 10% và margin-left 5% Theo Width Cha**



(→ Điểm cốt lõi: Cả `padding-top` (50px) lẫn `margin-left` (25px) đều lấy 500px chiều rộng cha làm mốc tính toán).

c) Chuyên Đề Chi Tiết: Thuộc Tính font-size Với Đơn Vị Phần Trăm (%):

Tổng hợp chi tiết về việc sử dụng `font-size` với đơn vị phần trăm (`%`) trong CSS, đặc biệt phân tích rõ cơ chế hoạt động khi phần tử cha không cài đặt `font-size`:

[Khái Niệm] 1. Cách Hoạt Động Cốt Lõi Của font-size: %

- Đơn vị `%` trong `font-size` là **tỉ lệ so với font-size của phần tử cha trực tiếp**.
- Nếu phần tử cha không khai báo `font-size`, nó sẽ **tự động kế thừa ngược lên trên**, cho đến khi gặp phần tử có `font-size` (thường là thẻ gốc `<html>`).
- Nếu xuyên suốt cây DOM không có phần tử nào khai báo, thì mặc định `font-size` chuẩn của trình duyệt cho `<html>` là **16**px.

[Bảng] 2. Bảng Ma Trận: Khi Nào font-size: % Hướng Về Thẻ <html>?

Tình Huống Trong DOM	Hướng Về <html>?	Giải Thích Cơ Chế Kế Thừa
Không có phần tử cha nào cài đặt font-size	[CÓ]	Tự động kế thừa thẳng lên thẻ gốc và lấy font-size mặc định của <html> (16px).
Phần tử cha gần nhất có cài đặt font-size	[KHÔNG]	Sẽ lấy trực tiếp giá trị của cha gần nhất làm điểm tựa cơ sở để tính %.
Nhiều lớp cha lồng nhau, mỗi lớp đều dùng %	[KHÔNG]	Mỗi lớp sẽ nhân dồn theo từng cấp (Hiệu ứng Cascading Compounding), dễ gây sai lệch kích thước.

3. Ví Dụ Minh Họa & Đo Lường Trực Quan Trên Chrome DevTools:

Trường Hợp 1: Không Có Phần Tử Cha Thiết Lập font-size:

Mã Nguồn: Thẻ Con Dùng font-size: 150% Khi Cha Không Set Cỡ Chữ

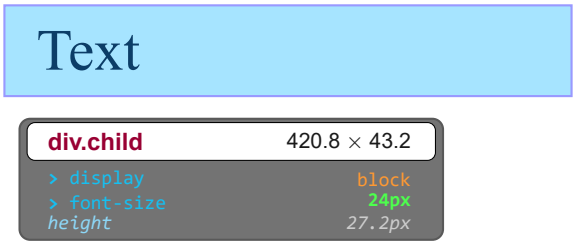
```
<!-- Khong co phan tu cha nao khai bao font-size -->
<div class="child" style="font-size: 150%;">Text</div>
```

[Khái Niệm] Công Thức Tính Toán Trường Hợp 1

Trình duyệt kế thừa cỡ chữ gốc của <html> (16px):

$$\text{font-size (.child)} = 150\% \times 16\text{px} = \left(\frac{150}{100}\right) \times 16\text{px} = \mathbf{24}\text{px}$$

BROWSER PREVIEW **Mô Phỏng Chrome DevTools: font-size = 24px (Khi Cha Không Cài Đặt)**



(→ Kết quả DevTools Inspector: Kích thước chữ thực tế đạt chuẩn 24px do lấy mốc 16px của <html>).

Trường Hợp 2: Phần Tử Cha Có Thiết Lập font-size: 20px:

Mã Nguồn: Thẻ Con Dùng font-size: 150% Khi Cha Có font-size: 20px

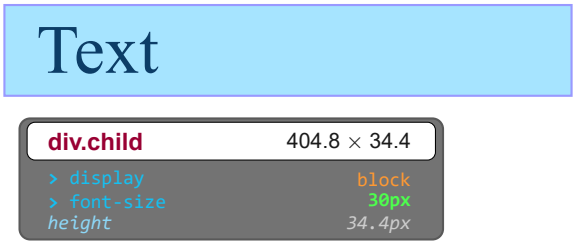
```
<div class="parent" style="font-size: 20px;">
  <div class="child" style="font-size: 150%;">Text</div>
</div>
```

[Khái Niệm] Công Thức Tính Toán Trường Hợp 2

Trình duyệt lấy trực tiếp cỡ chữ của khối cha `.parent` (20px) làm điểm tựa:

$$\text{font-size (.child)} = 150\% \times 20\text{px} = \left(\frac{150}{100}\right) \times 20\text{px} = \mathbf{30}\text{px}$$

BROWSER PREVIEW Mô Phỏng Chrome DevTools: font-size = 30px (Khi Cha Có font-size: 20px)



(→ Kết quả DevTools Inspector: Kích thước chữ thực tế đạt chuẩn **30**px do lấy mốc 20px của `.parent`).

4. Bảng Khuyến Nghị Lựa Chọn Đơn Vị Typography (Best Practices):

[Bảng] Bảng Hướng Dẫn Chọn Đơn Vị Kích Thước Chữ Trong Thực Tế

Mục Đích Thiết Kế Giao Diện	Nên Dùng Đơn Vị	Lý Do & Lợi Ích Kỹ Thuật
Kế thừa linh hoạt theo cha gần nhất	`%`	Cỡ chữ tự co giãn theo tỷ lệ của khối bao bọc.
Đồng bộ với kích thước gốc toàn trang	`rem` (root em)	Tránh hoàn toàn lỗi nhân dồn nhiều cấp, dễ quản lý trên toàn bộ hệ thống.
Tương đối theo chính phần tử con	`em`	Rất thích hợp cho `padding`, `margin`, bo góc của Component co giãn.

5. Mẹo Cài Đặt Nhất Quán (Consistent Typography Pattern):

Mã Nguồn Mẫu Thiết Lập Typography Nhất Quán

```
/* 1. Thiet lap font-size goc toan trang: */
html {
  font-size: 16px; /* Goc chung toan bo website */
}

/* 2. Dinh kieu cac the noi dung: */
p {
  font-size: 150%;  /* = 24px neu phan tu cha truc tiep la html */
  /* Hoac su dung rem de dam bao tuyet doi: */
  font-size: 1.5rem; /* Luon luon = 24px bat ke phan tu cha la gi */
}
```

d) Với line-height (chiều cao dòng):

- **Điểm tham chiếu**: Giá trị `font-size` của chính phần tử đó.
- **Cách hoạt động**: Chiều cao dòng bằng tỷ lệ phần trăm của kích thước chữ của chính nó.

Khai Báo line-height Theo %

```
p {
    font-size: 18px;
    line-height: 150%; /* 150% cua 18px = (150 / 100) * 18px = 27px */
}
```

BROWSER PREVIEW **Mô Phỏng Trực Quan line-height: 150% (Chiều Cao Dòng 27px Với Cỡ Chữ 18px)**

Dòng thứ nhất: Đoạn văn bản có phông chữ 18px và chiều cao dòng 150% (27px).
Dòng thứ hai: Khoảng cách giữa các dòng được giãn đều đặn, thông thoáng và dễ đọc trên mọi thiết bị.

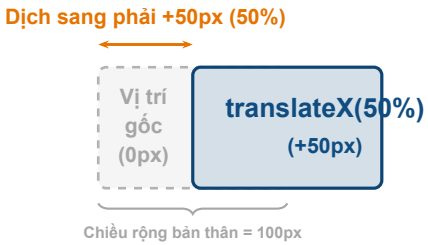
e) Với transform (biến đổi hình học):

- **Điểm tham chiếu**: Kích thước của chính phần tử đang được biến đổi.
- **Cách hoạt động**: Giá trị dịch chuyển (`translateX()`, `translateY()`), phóng to... sẽ dựa trên kích thước của bản thân phần tử đó.

Khai Báo transform: translateX(50%) Dựa Trên Kích Thước Bản Thân

```css
.box {
    width: 100px;
    height: 100px;
    transform: translateX(50%); /* Dich sang phai 50% cua width (100px) = 50px */
}
```

BROWSER PREVIEW Mô Phỏng Trực Quan transform: translateX(50%) Dịch Sang Phải 50px



(→ Kết quả: Phần tử tự dịch sang phải một khoảng bằng đúng 50% chiều rộng của chính nó (50px)).

4. Ưu Điểm Nổi Bật Của Đơn Vị Phần Trăm (%):

[Bảng] Bảng Tổng Hợp Ưu Điểm Của Đơn Vị % Trong Thiết Kế Web

Ưu Điểm Nổi Bật	Phân Tích Chi Tiết & Lợi Ích Thực Tế
Responsive Design (Thiết kế đáp ứng)	Ưu điểm lớn nhất. Khi kích thước trình duyệt hoặc phần tử cha thay đổi, phần tử tự động co giãn theo, thích ứng mượt mà trên desktop, tablet và mobile.
Fluid Layouts (Bố cục linh hoạt)	Tạo ra các bố cục dạng "dòng chảy" mềm mại, tự lấp đầy không gian có sẵn, tận dụng tối đa không gian màn hình.
Giảm thiểu Media Queries	Giảm đáng kể số lượng câu lệnh `@media` phải viết, giúp mã nguồn CSS gọn gàng và dễ bảo trì hơn.

5. Nhược Điểm Và Những Điều Cần Lưu Ý Khi Sử Dụng %:

[Bảng] Bảng Nhược Điểm Và Cân Nhắc Kỹ Thuật Khi Dùng Đơn Vị %

Nhược Điểm / Cân Nhắc	Phân Tích Chi Tiết & Rủi Ro Cần Tránh
Tính kế thừa phức tạp (Cascade)	Việc lồng ghép nhiều phần tử con liên tiếp dùng `%` dễ gây hiện tượng "phình to" hoặc "teo nhỏ" không mong muốn nếu không tính toán kỹ.
Khó kiểm soát chính xác	Khi cần kích thước cố định tuyệt đối (như đường viền, icon nhỏ), `%` không phải là lựa chọn tốt (nên dùng `px`).
Sự phụ thuộc vào phần tử cha	Phần tử cha phải có kích thước xác định. Nếu cha không set `height` rõ ràng, `height: 100%` ở con sẽ **hoàn toàn không có tác dụng**.
Vấn đề với typography	`rem` và `em` thường được ưu tiên hơn `%` cho kích thước chữ vì mang lại khả năng quản lý tỷ lệ phông chữ nhất quán hơn.

6. Khi Nào Nên Sử Dụng Đơn Vị % Trong Thực Tế?

[Bảng] Bảng Tổng Hợp Các Trường Hợp Ứng Dụng Thực Tế Của Đơn Vị %

Trường Hợp Thực Tế	Thuộc Tính CSS	Mục Đích Áp Dụng
Chia cột trong hệ thống lưới	`width: 33.33%, width: 49%`	Chia 3 cột đều nhau (33.33%) hoặc chia 2 cột (49%) kèm khoảng cách lề.
Khung bố cục chính (Container)	`width: 80%, margin: 0 auto`	Tạo khung trung tâm chiếm 80% độ rộng màn hình và tự động căn giữa.
Hình ảnh đáp ứng (Responsive Img)	`max-width: 100%, height: auto`	Đảm bảo ảnh không bao giờ tràn ra ngoài khung chứa và giữ đúng tỷ lệ khung hình.
Duy trì tỷ lệ khoảng cách	`padding: 5%`	Khoảng đệm tự co giãn tỷ lệ thuận với kích thước tổng thể của giao diện.

Mã Nguồn Mẫu Tổng Hợp Các Tình Huống Dùng Đơn Vị % Chuẩn Mực

```
<style>
/* 1. Thiet lap khung container trung tam: */
.container {
    width: 80%;       /* Chiem 80% chieu rong cua phan tu cha */
    margin: 0 auto;   /* Can giua khung */
}

/* 2. Chia cot co gian trong Grid/Layout: */
.column-3 {
    width: 33.33%;    /* 3 cot deu nhau */
    float: left;
}
.column-2 {
    width: 49%;       /* 2 cot co khoang cach */
    float: left;
    margin-right: 2%;
}

/* 3. Padding duy tri ty le: */
.card {
    padding: 5%;      /* Padding 5% dua tren width phan tu cha */
}

/* 4. Hinh anh Responsive linh hoat: */
img.responsive-img {
    max-width: 100%;  /* Anh khong bao gio vuot qua chieu rong khung cha */
    height: auto;     /* Tu dong giu dung ty le khung hinh */
}
</style>
```

BROWSER PREVIEW **Mô Phỏng Trực Quan Kết Quả Layout Chia Cột Và Ảnh Responsive (%)**

.container { width: 80%; margin: 0 auto; }

CỘT 1 (width: 49%)

Bài Viết Nổi Bật

Thẻ Card áp dụng padding 5% co giãn mượt mà theo độ rộng cha.

ĐỌC TIẾP

CỘT 2 (width: 49%)

img.responsive-img

max-width: 100%; height: auto;

Ảnh co giãn linh hoạt, không bao giờ tràn mép khung.

[Ghi Chú] Tóm Tắt Cốt Lõi Về Đơn Vị Phần Trăm (%)

Tóm lại, đơn vị `%` là một công cụ không thể thiếu trong CSS hiện đại, giúp bạn tạo ra các thiết kế linh hoạt và đáp ứng. Hiểu rõ **điểm tham chiếu của nó cho từng thuộc tính CSS** là chìa khóa để sử dụng nó một cách hiệu quả và tránh những lỗi bố cục không mong muốn.

7. Chuyên Đề Lưu Ý Thực Tế: Cơ Chế Hoạt Động Của height Khi Dùng Đơn Vị % (Vấn Đề HTML, Body & Khối Con)

Một trong những cạm bẫy kinh điển và thường xuyên gây bối rối nhất cho các lập trình viên Frontend là: **Nếu không cài đặt chiều cao (`height`) cho phần tử cha, nhưng lại khai báo `height: %` cho phần tử con, thì chiều cao phần trăm sẽ KHÔNG HOẠT ĐỘNG như kỳ vọng.**

a) Vì Sao height: % Không Hoạt Động Khi Cha Không Có Chiều Cao?

[Cảnh Báo] Nguyên Nhân Cốt Lõi Khiến height: % Bị Vô Hiệu Hóa

- Trong CSS, giá trị `height: %` của phần tử con bắt buộc phải dựa vào **chiều cao đã được xác định rõ ràng (Explicit Height)** của phần tử cha.
- Nếu phần tử cha không có `height` cụ thể, trình duyệt sẽ không có cơ sở toán học để tính toán tỷ lệ phần trăm. Kết quả là phần tử con sẽ **co lại thành** 0 **hoặc chỉ vừa khít với nội dung bên trong nó**.

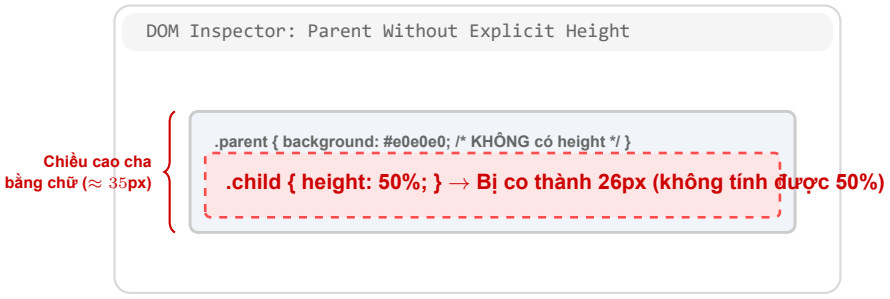
b) Ví Dụ Đối Chiếu Trực Quan: Không Hiệu Quả vs Hiệu Quả

Trường Hợp 1: KHÔNG HIỆU QUẢ – Phần Tử Cha Không Khai Báo height

```
<style>
.parent {
  background: #e0e0e0;
  /* KHONG co height xac dinh */
}
.child {
  height: 50%;   /* KHONG hoat dong dung! */
  background: #f48fb1;
}
</style>

<div class="parent">
  <div class="child">Noi dung Hello</div>
</div>
```

BROWSER PREVIEW — **Mô Phỏng: Cha Không Khai Báo height**



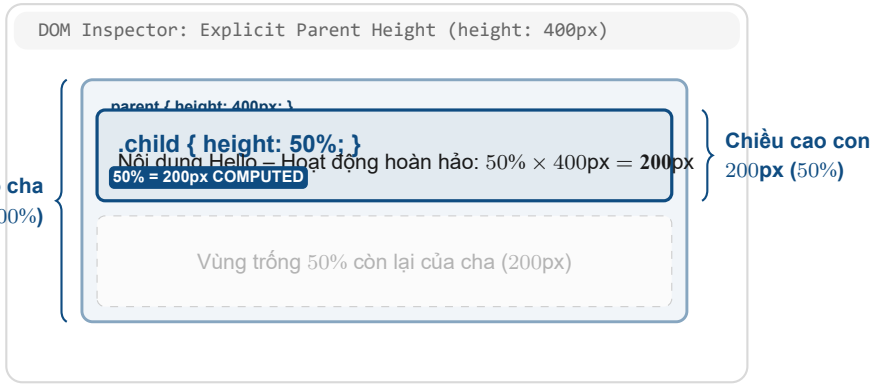
(→ **Thất bại**: .child không thể tính ra 50% vì .parent không có kích thước cụ thể để làm mốc).

Trường Hợp 2: HOẠT ĐỘNG HOÀN HẢO – Phần Tử Cha Có height Xác Định

```
<style>
.parent {
  height: 400px; /* Cha co chieu cao ro rang: 400px */
  background: #e0e0e0;
}
.child {
  height: 50%;   /* Hoat dong hoan hao: 50% cua 400px = 200px */
  background: #f48fb1;
}
</style>

<div class="parent">
  <div class="child">Noi dung Hello</div>
</div>
```

BROWSER PREVIEW — **Mô Phỏng: Cha Có height: 400px Chuẩn Mực**



(→ **Thành công**: Trình duyệt lấy $50\% \times 400\text{px} = 200\text{px}$ để gán chính xác cho khối con).

c) Bảng Hướng Dẫn Cách Xử Lý Chiều Cao Linh Hoạt Trong Thực Tế:

[Bảng] Bảng Giải Pháp Xử Lý Khi Cần Chiều Cao Tương Đối

Tình Huống Giao Diện	Cách Xử Lý Chuẩn Mực
Cần dùng % cho height	Đặt chiều cao rõ ràng cho phần tử cha (bằng px, vh, v.v.).
Không muốn dùng px cố định	Dùng min-height, mô hình **Flexbox**, **CSS Grid**, hoặc đơn vị Viewport (vh, ví dụ: height: 50vh).

[Mẹo] Tóm Tắt 3 Phương Án Thay Thế Khi Không Thể Dùng height: %

1. **Sử dụng Flexbox hoặc Grid**: Tự động kéo dãn phần tử con theo chiều cao cột cha bằng thuộc tính `align-items: stretch`.
2. **Sử dụng đơn vị Viewport (vh)**: Ví dụ `height: 50vh;` (bằng đúng 50% chiều cao màn hình hiển thị), hoàn toàn độc lập với phần tử cha.
3. **Sử dụng `min-height`**: Kết hợp co giãn tự nhiên theo nội dung mà vẫn đảm bảo độ cao tối thiểu cho khung giao diện.

d) Vấn Đề Kinh Điển: Dùng height: 100% Với Thẻ <body> Và <html>

Khi bạn muốn một khối con chiếm trọn vẹn 100% chiều cao màn hình trình duyệt bằng đơn vị phần trăm, việc chỉ thiết lập `height: 100%;` cho thẻ `<body>` hoặc khối con là **hoàn toàn không đủ**:

Lỗi Thường Gặp: Chỉ Đặt height: 100% Cho Khối Con Hoặc Body

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body {
      /* KHONG co height ro rang cho html -> body khong co co so chieu cao */
      background: #e0e0e0;
    }
    .child {
      height: 100%;  /* KHONG co tac dung chiem tron man hinh! */
      background: #f48fb1;
    }
  </style>
</head>
<body>
  <div class="child">Child khong the chiem 100% chieu cao man hinh</div>
</body>
</html>
```

[Khái Niệm] Giải Thích Cơ Chế Chuỗi Kế Thừa (Chain of Inheritance)

Khối `.child` không chiếm 100% chiều cao màn hình vì `<body>` chưa có chiều cao xác định, mà bản thân `<body>` lại phụ thuộc vào thẻ gốc `<html>`.

Cách Làm Đúng Chuẩn: Thiết Lập height: 100% Đồng Thời Cho Cả html Và body

```html
<!DOCTYPE html>
<html>
<head>
  <style>
    /* 1. Dat height: 100% cho ca html va body: */
    html, body {
      height: 100%;   /* Bat buoc phai dat cho ca hai the */
      margin: 0;      /* Xoa margin mac dinh 8px cua trinh duyet */
    }

    /* 2. Bay gio khoi con co the dung height: 100% hoan hao: */
    .child {
      height: 100%;   /* Chiem dung 100% chieu cao body (toan bo viewport) */
      background: #f48fb1;
    }
  </style>
</head>
<body>
  <div class="child">Chiem toan bo chieu cao man hinh</div>
</body>
</html>
```

BROWSER PREVIEW So Sánh: height: 100% Khi Chưa vs Đã Khai Báo Cho html



(→ Điểm then chốt: Để `height: 100%` phủ kín màn hình, chuỗi kế thừa từ `html` → `body` → `.child` đều phải được gán `height: 100%`).

e) Bảng Ma Trận So Sánh Hành Vi Của Thẻ <html> Và <body> Trong CSS:

[Bảng] Bảng Đối Chiếu Quy Tắc Chiều Cao Của Thẻ html Và body

Thẻ	Hành Vi Mặc Định	Khi Dùng height: 100%	Khi Dùng height: 100vh
`html`	Chiều cao tự giãn theo nội dung bên trong, không có giới hạn cứng.	Cần thiết để thẻ `body` có thể sử dụng `height: 100%`.	Không bắt buộc áp dụng, vì `vh` dựa trực tiếp trên Viewport.
`body`	Tự giãn theo nội dung, không tự động cao bằng toàn bộ màn hình.	Chiều cao bằng chiều cao của `<html>` (chỉ khi `<html>` có `height: 100%`).	Chiều cao bằng đúng chiều cao Viewport, hoàn toàn độc lập với thẻ `<html>`.

[Bảng] Bảng Tóm Tắt Quy Trình 3 Bước Khi Dùng % Chiều Cao Với Thẻ body

Thành Phần	Hành Động Bắt Buộc
Thẻ `<html>`	Khai báo `height: 100%;`
Thẻ `<body>`	Khai báo `height: 100%; margin: 0;`
Khối con `(.child)`	Bây giờ có thể sử dụng `height: %` hoặc `height: 100%;` một cách chuẩn xác.

1.5.4 Chuyên Đề Đơn Vị em (em Masterclass Trong CSS)

Đơn vị `em` là một trong những đơn vị đo lường tương đối lâu đời và quan trọng bậc nhất trong CSS, bắt nguồn từ thuật ngữ ngành in ấn (chiều rộng của ký tự chữ "M" hoa). Trong CSS hiện đại, `em` hoạt động linh hoạt theo cơ chế kế thừa ngữ cảnh phông chữ.

1. Khái Niệm Cốt Lõi & Bản Chất Đơn Vị em:

Đơn vị `em` là đơn vị tương đối, phụ thuộc trực tiếp vào `font-size` của phần tử cha gần nhất (hoặc kích thước phông chữ của chính nó nếu khai báo trực tiếp cho các thuộc tính kích thước khác).

[Khái Niệm] Bản Chất Quy Đổi Của Đơn Vị em

$$\mathbf{1}\text{em} = \mathbf{100}\%\ \text{font-size của phần tử hiện tại hoặc cha trực tiếp}$$

Công thức quy đổi giá trị thực tế:

$$\text{Giá Trị Thực Tế (px)} = \text{Giá Trị em} \times \text{font-size của phần tử cha}$$

2. Ví Dụ Đơn Giản: Kích Thước Chữ (font-size) Với Đơn Vị em:

Xét ví dụ phần tử cha chứa cỡ chữ 20px và thẻ con được gán kích thước 2em:

Ví Dụ Đơn Giản: Xác Định font-size Bằng Đơn Vị em

```
<div class="parent">
  <p class="child">Chữ này 2em</p>
</div>

<style>
  .parent {
    font-size: 20px;
  }
  .child {
    font-size: 2em; /* = 2 x 20px = 40px */
  }
</style>
```

BROWSER PREVIEW **Mô Phỏng Kết Quả Render: .child Có font-size Là 40px**



(→ Kết quả: Phần tử `.child` nhận kích thước `font-size` là 40px dựa trên cỡ chữ 20px của cha).

3. Ví Dụ Rõ Ràng Hơn: Áp Dụng em Cho padding Của Phần Tử:

Đơn vị `em` không chỉ áp dụng cho phông chữ mà còn được sử dụng rộng rãi cho các thuộc tính Box Model như `padding`, `margin`, `width`…Trong trường hợp này, 1em được tính dựa theo cỡ chữ của chính nó (hoặc kế thừa từ phần tử cha nếu phần tử con không khai báo `font-size`).

Ví Dụ Rõ Ràng Hơn: Áp Dụng em Cho Khoảng Đệm padding

```
<div class="outer">
  <div class="inner">Tôi có padding 2em</div>
</div>

<style>
  .outer {
    font-size: 20px;
  }
  .inner {
    padding: 2em; /* = 2 x 20px = 40px padding */
    background-color: lightcoral;
  }
</style>
```

BROWSER PREVIEW **Mô Phỏng Trực Quan: Khối .inner Nhận Khoảng Đệm padding: 2em = 40px**



(→ Kết quả: Phần tử `.inner` có khoảng đệm 40px cả 4 phía nhờ phép nhân 2×20px từ cỡ chữ kế thừa).

4. Lưu Ý Quan Trọng: 3 Tình Huống & Hành Vi Khi Sử Dụng em:

Khi làm việc với đơn vị em, các lập trình viên bắt buộc phải nắm rõ 3 tình huống hành vi đặc thù sau đây:

[Bảng] Bảng 3 Tình Huống & Hành Vi Đặc Thù Của Đơn Vị em

Tình Huống	Hành Vi Cụ Thể
Lồng nhiều cấp	`em` sẽ **nhân chồng (compounding)** qua từng tầng → có thể khiến kích thước phông chữ bị phóng đại hoặc teo nhỏ một cách khó kiểm soát.
Áp dụng cho padding, margin, width…	Cũng hoạt động tương tự: Độ dài được tính tỷ lệ theo `font-size` của chính phần tử đó.
Không đặt font-size cụ thể cho cha	`em` mặc định sẽ dựa vào kích thước phông chữ thiết lập của trình duyệt (thông thường tiêu chuẩn là 16px).

5. So Sánh Nhanh Yếu Tố Tham Chiếu Của 4 Đơn Vị Cốt Lõi:

Dưới đây là bảng tổng hợp nhanh điểm tựa tham chiếu của 4 đơn vị phổ biến nhất trong CSS:

[Bảng] Bảng So Sánh Nhanh Điểm Tựa Tham Chiếu: em, rem, %, px

Đơn Vị	Dựa Vào (Điểm Tựa Tham Chiếu)
`em`	`font-size` của phần tử cha gần nhất (hoặc chính nó đối với thuộc tính Box Model).
`rem`	`font-size` của thẻ gốc `<html>` duy nhất trên toàn bộ trang.
`%`	Phụ thuộc vào thuộc tính cụ thể (ví dụ: `width` theo chiều rộng cha, `font-size` theo cỡ chữ cha).
`px`	Cố định, không bị ảnh hưởng bởi bất kỳ phần tử nào khác.

6. Quy Tắc Hoạt Động Cốt Lõi Của Đơn Vị em:

Điểm tham chiếu của đơn vị `em` phụ thuộc trực tiếp vào thuộc tính CSS mà nó được áp dụng:

- **Khi áp dụng cho thuộc tính `font-size`**: 1em bằng đúng kích thước `font-size` của **phần tử cha trực tiếp**.
- **Khi áp dụng cho các thuộc tính kích thước khác** (`padding`, `margin`, `width`, `border-radius`...): 1em bằng đúng kích thước `font-size` của **chính phần tử đó**.

[Bảng] Bảng Điểm Tham Chiếu Của Đơn Vị em

Thuộc Tính Áp Dụng	Điểm Tựa Tham Chiếu	Công Thức Tính Toán Độ Dài Thực Tế
`font-size`	`font-size` của phần tử cha	Cỡ chữ thực tế = Giá trị em × font-size của cha.
`padding, margin, width, border-radius`	`font-size` của chính nó	Kích thước thực tế = Giá trị em × font-size của chính phần tử.

7. Cạm Bẫy Cần Tránh: Hiệu Ứng Nhân Dồn Kích Thước (Em Compounding Effect):

Khi bạn lồng ghép nhiều cấp phần tử con liên tiếp và mỗi cấp đều sử dụng `font-size` bằng đơn vị `em`, kích thước chữ sẽ bị **nhân dồn qua từng tầng**, dẫn đến việc phần tử càng lồng sâu thì chữ càng phình to hoặc teo nhỏ một cách khó kiểm soát.

Minh Họa Hiện Tượng Nhân Dồn Kích Thước Với Danh Sách Menu Lồng Nhau

```
<style>
/* Co chu goc cua danh sach: 16px */
ul.compounding-menu {
  font-size: 16px;
}

/* Moi cap danh sach con deu tang kich thuoc them 1.2em */
ul.compounding-menu li {
  font-size: 1.2em; /* 1.2 * font-size cua ul cha truc tiep */
}
</style>

<ul class="compounding-menu">
  <li>Cap 1 (16px * 1.2 = 19.2px)
    <ul>
      <li>Cap 2 (19.2px * 1.2 = 23.04px)
        <ul>
          <li>Cap 3 (23.04px * 1.2 = 27.65px - Bi phinh to qua muc!)</li>
        </ul>
      </li>
    </ul>
  </li>
</ul>
```

[Khái Niệm] Phân Tích Toán Học Phép Nhân Dồn Đơn Vị em

- **Cấp 1**: $1.2\text{em} \times 16\text{px} = \mathbf{19.2}\text{px}$.
- **Cấp 2**: $1.2\text{em} \times 19.2\text{px} = \mathbf{23.04}\text{px}$.
- **Cấp 3**: $1.2\text{em} \times 23.04\text{px} = \mathbf{27.65}\text{px}$ (gần gấp đôi kích thước phông chữ gốc ban đầu).

BROWSER PREVIEW Mô Phỏng Trực Quan Hiệu Ứng Nhân Dồn Khi Dùng em Cho Danh Sách Lồng

- **Cấp 1 (19.2px): Menu cha tiêu chuẩn**
  - **Cấp 2 (23.04px): Chữ bắt đầu to hơn rõ rệt**
    - **Cấp 3 (27.65px): Chữ bị phình to ngoài ý muốn!**

8. Ứng Dụng Thực Tế Tuyệt Vời Của em: Nút Bấm Co Giãn Tự Động (Scalable Buttons):

Mặc dù `em` phức tạp cho `font-size`, nhưng nó lại là **vũ khí tối thượng để thiết kế các thành phần UI có khả năng tự động co giãn tỷ lệ** (**Scalable Components**). Khi sử dụng `em` cho `padding` và `border-radius` của nút bấm, khoảng đệm sẽ tự động mở rộng hài hòa khi bạn thay đổi cỡ chữ mà không cần viết thêm bất kỳ dòng CSS nào!

Mã Nguồn Nút Bấm Co Giãn Tỷ Lệ Hoàn Hảo Với Đơn Vị em

```
<style>
/* Khung nut bam co ban: Padding va Radius dua theo font-size cua chinh no */
.btn {
  font-family: inherit;
  font-weight: bold;
  padding: 0.5em 1.2em;      /* 0.5em tren/duoi, 1.2em trai/phai */
  border-radius: 0.35em;     /* Bo goc ty le theo co chu */
  background: #0f4c81;
  color: white;
  border: none;
  cursor: pointer;
}

/* Nut nho (Small): */
.btn-sm { font-size: 12px; }  /* Padding tu tinh: 6px 14.4px, Radius: 4.2px */

/* Nut tieu chuan (Medium): */
.btn-md { font-size: 16px; }  /* Padding tu tinh: 8px 19.2px, Radius: 5.6px */

/* Nut lon (Large): */
.btn-lg { font-size: 22px; }  /* Padding tu tinh: 11px 26.4px, Radius: 7.7px */
</style>

<button class="btn btn-sm">Small Button</button>
<button class="btn btn-md">Medium Button</button>
<button class="btn btn-lg">Large Button</button>
```

BROWSER PREVIEW Mô Phỏng Trực Quan Bộ Nút Bấm Tự Co Giãn Tỷ Lệ Theo Cỡ Chữ (Scalable Buttons)



(→ Khoảng cách Padding và Bo góc tự động nở rộng tỷ lệ thuận theo cỡ chữ mà không cần viết lại thuộc tính padding).

Cách Hoạt Động Của Đơn Vị em Qua 9 Trường Hợp Thực Chiến

Để làm chủ hoàn toàn đơn vị `em` và loại bỏ mọi bối rối khi thiết kế giao diện, chúng ta cùng phân tích chi tiết cách vận hành toán học của `em` qua 9 kịch bản điển hình từ cơ bản đến phức tạp:

Trường Hợp 1: Khi Cả Cha Và Con Đều Không Khai Báo font-size:

- **Ngữ cảnh**: Khi không khai báo `font-size` cho cả phần tử cha và phần tử con, trình duyệt sẽ sử dụng giá trị mặc định của thẻ gốc `<html>` (thường là 16px).
- **Mặc định trình duyệt**: Thiết lập mặc định của thẻ `<html>` là 16px.

Mặc Định Của Trình Duyệt Cho Thẻ Gốc html

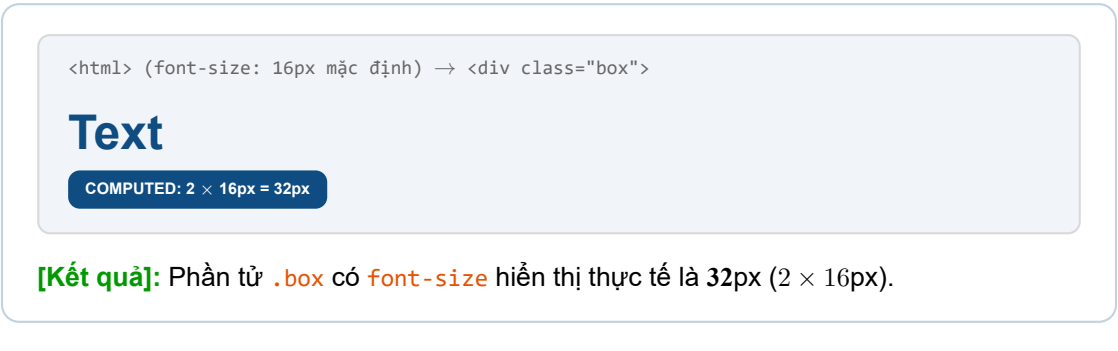
```
html {
  font-size: 16px;
}
```

Ví Dụ 1.1: Không Khai Báo font-size Cho Bất Kỳ Phần Tử Nào

```
<div class="box">Text</div>

<style>
  .box {
    font-size: 2em; /* 2 x 16px (mac dinh html) = 32px */
  }
</style>
```

BROWSER PREVIEW — **Kết Quả: .box Nhận font-size Là 32px**



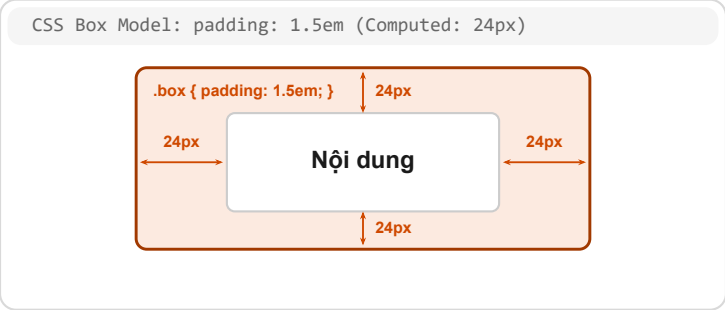
[Kết quả]: Phần tử `.box` có `font-size` hiển thị thực tế là **32**px (2×16px).

Ví Dụ 1.2: Dùng em Trong padding Khi Không Có font-size Khai Báo

```
<div class="box">Noi dung</div>

<style>
  .box {
    padding: 1.5em; /* 1.5 x 16px = 24px padding moi phia */
    background-color: yellow;
    border: 1px solid black;
  }
</style>
```

BROWSER PREVIEW — Kết Quả: .box Có padding 24px Xung Quanh



(→ Kết quả: `.box` có khoảng đệm 24px xung quanh, dựa vào `font-size` mặc định của trình duyệt là 16px).

[Bảng] Bảng Tổng Kết Điểm Tham Chiếu Của em Trong Trường Hợp 1

Tình Huống	em Tham Chiếu Đến Đâu?
Không có font-size cho cha và con	Sẽ dùng `font-size` mặc định của thẻ `<html>` là 16px.
Có font-size ở cha	`em` sẽ nhân trực tiếp với giá trị đó.
Có nhiều cấp em lồng nhau	Có thể gây nhân chồng, dẫn đến kích thước phóng đại không kiểm soát.

[Mẹo] Gợi Ý Kiểm Soát em & Kỹ Thuật 62.5% Kinh Điển

Nếu muốn `em` ổn định, bạn có thể đặt `font-size` rõ ràng cho phần tử cha hoặc dùng `rem` để tham chiếu về gốc. Để đảm bảo tính nhất quán và dễ nhẩm toán, nhiều lập trình viên CSS thường áp dụng:

```
html {
  font-size: 62.5%; /* 62.5% x 16px = 10px -> 1em = 10px rat de tinh toan! */
}
```

Trường Hợp 2: Phần Tử Con Có font-size Là px Và Dùng em Cho Thuộc Tính Box Model (width, height, padding, margin):

- **Quy tắc cơ bản**: Nếu `font-size` của phần tử được đặt bằng `px`, thì các thuộc tính khác (`width`, `height`, `padding`, `margin`…) viết bằng `em` sẽ **dựa trên chính font-size của phần tử đó** (hoàn toàn độc lập với phần tử cha).
- **Cách tính**:

$$\text{Giá trị tính bằng px} = \text{Giá trị em} \times \text{font-size của chính phần tử}$$

Ví Dụ 2: Xác Định Box Model Bằng em Theo font-size Của Chính Nó

```
<div class="box">Noi dung</div>

<style>
  .box {
    font-size: 20px;      /* Co so: 1em = 20px */
    width: 10em;          /* 10 x 20px = 200px */
    height: 5em;          /* 5 x 20px = 100px */
    padding: 1em;         /* 1 x 20px = 20px */
    background-color: lightblue;
  }
</style>
```

[Bảng] Bảng Tính Toán Chi Tiết Thuộc Tính Box Model Theo em

Thuộc Tính	Giá Trị em	Tính Theo font-size: 20px	Kết Quả px Thực Tế
`width`	`10em`	10×20**px**	**200**px
`height`	`5em`	5×20**px**	**100**px
`padding`	`1em`	1×20**px**	**20**px (mỗi phía)

[Ghi Chú] 3 Lưu Ý Cốt Lõi Khi Dùng Box Model Với em

1. Nếu `font-size` được đặt bằng `px` → `em` dựa trên chính giá trị đó, **không liên quan đến phần tử cha**.
2. Đây là điểm khác biệt rõ ràng với `em` khi dùng cho `font-size`, vốn dựa theo cha.
3. Nếu bạn thay đổi `font-size`, tất cả giá trị `em` liên quan sẽ tự động co giãn theo → cực kỳ thuận tiện cho thiết kế Responsive.

[Bảng] So Sánh Nhanh Điểm Tựa Tham Chiếu Của 1em Trong Trường Hợp 2

Trường Hợp Khai Báo	1em Dựa Trên Cơ Sở Nào?
`font-size: 20px`	20px (của chính phần tử đó).
`font-size: 2em`	Nhân với `font-size` của phần tử CHA.
`Không khai báo font-size`	Mặc định 16px (kế thừa từ thẻ gốc `<html>`).

Trường Hợp 3: Phần Tử Cha Có font-size, Phần Tử Con Không Khai Báo:

- **Cách hoạt động**: Nếu phần tử con không có `font-size`, nó sẽ tự động kế thừa giá trị `font-size` từ phần tử cha gần nhất có khai báo.
- **Bản chất**: Đây là tính kế thừa tự động (**CSS Inheritance**) — vì `font-size` là một thuộc tính có khả năng kế thừa theo phân cấp DOM.

Ví Dụ 3: Phần Tử Con Kế Thừa Tự Động font-size Từ Phần Tử Cha

```
<div class="parent">
  <p class="child">Noi dung cua con</p>
</div>

<style>
  .parent {
    font-size: 24px; /* Cha co font-size */
  }
  .child {
    /* KHONG khai bao font-size */
  }
</style>
```

BROWSER PREVIEW Kết Quả: .child Tự Động Nhận font-size Là 24px Từ .parent

Nội dung của con (thừa hưởng chính xác cỡ chữ 24px từ phần tử cha)

[Kết luận]: Phần tử `.child` sẽ tự động nhận `font-size: 24px` từ `.parent` vì `.child` không có `font-size`, nó kế thừa từ `.parent`.

[Bảng] Bảng Tổng Quát Cơ Chế Kế Thừa font-size Giữa Cha Và Con

Phần Tử Cha	Phần Tử Con	Kết Quả Thực Tế Ở Phần Tử Con
`font-size: 24px`	(không có)	Con = **24**px
`font-size: 20px`	(không có)	Con = **20**px
(không có)	(không có)	Con = **16**px (lấy mặc định từ thẻ gốc `<html>`)

[Mẹo] Ứng Dụng Thực Tế Của Tính Kế Thừa font-size

Thuộc tính `font-size` luôn kế thừa theo mặc định nếu không khai báo cụ thể. Đây là cách dễ kiểm soát toàn bộ hệ thống chữ trong một layout: Chỉ cần đặt `font-size` ở phần tử bao bọc lớn (ví dụ: `body`, `.container`, `section`), và để tất cả các phần tử con tự động thừa kế.

Trường Hợp 4: Phần Tử Cha Có font-size, Phần Tử Con Dùng Đơn Vị em:

Nếu phần tử con sử dụng `em` cho cả `font-size` lẫn các thuộc tính kích thước khác (`width`, `padding`…):

1. `font-size: x em` → dựa vào `font-size` của **phần tử cha**.
2. Các thuộc tính khác (`width`, `height`, `padding`, `margin`) dùng `em` → dựa trên `font-size` của **chính phần tử con đó** (giá trị đã được tính toán xong từ cha).

Ví Dụ 4: Cha Có font-size, Con Dùng em Cho Cả Chữ Lẫn Box Model

```
<div class="parent">
  <div class="child">Noi dung</div>
</div>

<style>
  .parent {
    font-size: 20px;   /* 1em = 20px */
  }
  .child {
    font-size: 1.5em;  /* 1.5 x 20px = 30px */
    padding: 1em;      /* 1 x 30px = 30px (vi font-size cua .child la 30px) */
    width: 10em;       /* 10 x 30px = 300px */
    background-color: lightgreen;
  }
</style>
```

[Bảng] Bảng Phân Tích Tính Toán Chi Tiết Thuộc Tính Trong Ví Dụ 4

Thuộc Tính	Điểm Tựa Dựa Trên	Tính Ra Độ Dài Thực Tế (px)
`font-size: 1.5em`	1.5×20px (của .parent)	$=$ 30px
`padding: 1em`	1×30px (của chính nó)	$=$ 30px
`width: 10em`	10×30px (của chính nó)	$=$ 300px

[Bảng] Bảng Tổng Kết Quy Tắc em Khi Cả Chữ Và Box Model Cùng Dùng em

Thuộc Tính Dùng em	Dựa Theo font-size Của Ai?
`font-size: em`	Phần tử cha gần nhất.
`padding, margin, width, height`	Phần tử chính nó (sau khi đã tính toán xong `font-size`).

[Cảnh Báo] Lưu Ý Về Tính Kế Thừa Nhân Dồn

Đơn vị `em` có tính kế thừa theo cấp, nên nếu lồng nhiều cấp `font-size: 1.2em`, kích thước sẽ bị phóng đại dần. Để tránh sự phức tạp không mong muốn, bạn có thể chuyển sang dùng đơn vị `rem` để luôn luôn bám vào thẻ gốc `<html>`.

Trường Hợp 5: Cả Cha Và Con Đều Đặt font-size Bằng px, Thuộc Tính Khác Dùng em:

- **Nguyên tắc hoạt động**: Các thuộc tính như `width`, `height`, `padding`, `margin` khi dùng `em` sẽ **luôn luôn dựa trên font-size của chính phần tử đó** (hoàn toàn không quan tâm đến `font-size` của phần tử cha).
- **Điểm then chốt**: Không quan trọng `font-size` đó được khai báo bằng `px`, `em` hay `rem`…miễn là đã được xác định cụ thể trên phần tử đó.

Ví Dụ 5: Cả Cha Và Con Đều Đặt font-size px, Box Model Dùng em

```html
<div class="parent">
  <div class="child">Noi dung con</div>
</div>

<style>
  .parent {
    font-size: 20px;
  }
  .child {
    font-size: 30px;       /* font-size rieng cua con */
    width: 10em;           /* = 10 x 30px = 300px */
    padding: 1em;          /* = 1 x 30px = 30px */
    margin: 0.5em;         /* = 0.5 x 30px = 15px */
    background: lightcoral;
  }
</style>
```

[Bảng] Bảng Phân Tích Tính Toán Cho Phần Tử .child Trong Ví Dụ 5

Thuộc Tính .child	Đơn Vị em Tính Theo Gì?	Tính Ra Độ Dài Thực Tế (px)
`font-size: 30px`	Do bạn trực tiếp thiết lập	**30**px
`width: 10em`	$10 \times$ font-size của chính nó	10×30px $=$ **300**px
`padding: 1em`	$1 \times$ font-size của chính nó	1×30px $=$ **30**px
`margin: 0.5em`	$0.5 \times$ font-size của chính nó	0.5×30px $=$ **15**px

[Bảng] Bảng Tổng Kết Nguyên Tắc Phân Định Điểm Tham Chiếu

Thuộc Tính Dùng em	Dựa Vào font-size Của Ai?
`font-size`	Phần tử cha gần nhất.
`width, height, padding, margin`	Phần tử chính nó (sau khi đã tính xong `font-size`).

[Ghi Chú] Lưu Ý Thiết Lập Cơ Sở Đo Lường

Khi bạn gán `font-size` cụ thể (dù bằng `px`, `em` hay `rem`) → các thuộc tính khác dùng `em` sẽ lấy đúng giá trị đó làm cơ sở. Để việc tính toán luôn chuẩn xác, hãy luôn đặt rõ ràng `font-size` cho phần tử nếu bạn dự định dùng `em` để thiết lập kích thước khác cho nó.

Trường Hợp 6: Thẻ Cha Có font-size Tương Đối (%, em) Và Cơ Chế Kế Thừa:

1. **font-size tương đối là gì?** Là khi bạn dùng các đơn vị tương đối như `em`, `%`, `rem` thay vì `px`. Ví dụ: `.parent { font-size: 120%; }` (tương đối theo phần tử cha của nó).
2. **Khi thẻ cha dùng đơn vị tương đối nhưng không có phần tử cấp trên đặt font-size**: Trình duyệt sẽ lấy kích thước mặc định của thẻ gốc `<html>` làm điểm tựa (thường là 16px). Tức là:

$$\text{font-size: } 120\% \quad \rightarrow \quad 120\% \times 16\text{px} = \mathbf{19.2}\text{px}$$

3. **Thẻ con kế thừa như thế nào?** Thẻ con dùng đơn vị `%` hoặc `em` sẽ kế thừa dựa trên giá trị `font-size` đã tính toán xong của thẻ cha.

Ví Dụ 6: Cha Dùng Tỷ Lệ Phần Trăm, Con Dùng Đơn Vị em

```
<div class="parent">
  <p class="child">Noi dung</p>
</div>

<style>
  .parent {
    font-size: 125%;     /* = 1.25 x 16px = 20px */
  }
  .child {
    font-size: 1.5em;    /* = 1.5 x 20px = 30px */
  }
</style>
```

[Bảng] Bảng Tổng Kết Quy Đổi font-size Qua 3 Cấp Trong Ví Dụ 6

Cấp Độ	font-size Tính Được	Ghi Chú & Điểm Tựa
`html`	16px (mặc định)	Cơ sở gốc chuẩn của trình duyệt.
`.parent`	125% = 20px	So với giá trị mặc định của `html` (1.25 × 16px).
`.child`	1.5em = 30px	So với giá trị đã tính của `.parent` (1.5 × 20px).

Trường Hợp 7: Thứ Tự Ưu Tiên Kế Thừa font-size Khi Dùng em (3 Cấp Ưu Tiên):

Khi một phần tử sử dụng đơn vị `em`, trình duyệt sẽ duyệt qua quy tắc thứ tự ưu tiên 3 cấp sau đây để xác định kích thước:

[Bảng] Bảng Thứ Tự Ưu Tiên Kế Thừa font-size Khi Dùng em

Ưu Tiên	Mức Kế Thừa	Giải Thích Cơ Chế
1	Chính phần tử đó	Nếu phần tử có khai báo `font-size`, thì các thuộc tính `em` khác sẽ tính dựa trên chính phần tử đó.
2	Thẻ cha trực tiếp	Nếu phần tử không có `font-size`, nó sẽ kế thừa từ thẻ cha gần nhất.
3	Gốc (thẻ `<html>`)	Nếu cả phần tử và phần tử cha đều không có `font-size`, trình duyệt sẽ dùng giá trị mặc định từ `<html>` (thường là 16px).

Ví Dụ 7: Minh Họa 3 Cấp Lồng Nhau Tăng Lũy Tiến Kích Thước

```
<div class="grandparent">
  <div class="parent">
    <p class="child">Text</p>
  </div>
</div>

<style>
  .grandparent { font-size: 16px; }
  .parent      { font-size: 1.5em; }   /* = 1.5 x 16px = 24px */
  .child       { font-size: 1.5em; }   /* = 1.5 x 24px = 36px */
</style>
```

[Khái Niệm] Tóm Tắt Bản Chất Cốt Lõi Về Đơn Vị em

`em` là đơn vị tương đối thông minh nhưng rất dễ gây lỗi kế thừa chồng lớp. Khi sử dụng `font-size: 1.5em` trong nhiều cấp, bạn cần hiểu rõ nó sẽ cộng dồn theo cha; nếu cha không xác định thì sẽ kế thừa từ gốc (`html`, thường là 16px). Nếu bạn muốn lấy kích thước chuẩn theo gốc mà không bị phụ thuộc vào thẻ cha, giải pháp lý tưởng là sử dụng đơn vị `rem` (**Root EM**), ví dụ: `font-size: 1.2rem;` (luôn tính trực tiếp từ thẻ `<html>`).

Trường Hợp 8: Mô Hình Thực Nghiệm 4 Cấp div Lồng Nhau (Ông – Bố – Con – Cháu):

Dưới đây là một ví dụ thực tế với 4 cấp `div` lồng nhau: Ông → Bố → Con → Cháu, tất cả đều dùng `font-size: 1.5em` để thấy rõ hiệu ứng kích thước bị phóng đại lũy tiến:

Ví Dụ 8: 4 Cấp div Lồng Nhau Với font-size Dùng em

```
<div class="grandparent">
  Ong (grandparent)
  <div class="parent">
    Bo (parent)
    <div class="child">
      Con (child)
      <div class="grandchild">
        Chau (grandchild)
      </div>
    </div>
  </div>
</div>

<style>
  .grandparent {
    font-size: 16px;
    color: white;
    background: #333;
    padding: 10px;
  }
  .parent {
    font-size: 1.5em; /* = 1.5 * 16px = 24px */
    background: #444;
    padding: 10px;
    margin-top: 10px;
  }
  .child {
    font-size: 1.5em; /* = 1.5 * 24px = 36px */
    background: #555;
    padding: 10px;
    margin-top: 10px;
  }
  .grandchild {
    font-size: 1.5em; /* = 1.5 * 36px = 54px */
    background: #666;
    padding: 10px;
    margin-top: 10px;
  }
</style>
```

[Bảng] Bảng Tính Toán Phóng Đại Lũy Tiến Qua 4 Cấp Lồng Nhau

Phần Tử	Công Thức Tính Toán	font-size Thực Tế (px)
`.grandparent`	Cố định trực tiếp	**16**px
`.parent`	1.5em × 16px	**24**px
`.child`	1.5em × 24px	**36**px
`.grandchild`	1.5em × 36px	**54**px (gấp hơn 3.3 lần kích thước ban đầu!)

[Mẹo] Cách Khắc Phục Nếu Không Muốn Chữ Bị Phình To

Để kiểm soát kích thước cho các phần tử lồng sâu mà không bị phóng đại theo cha, bạn có thể thiết lập kích thước cố định bằng `px` hoặc dùng `rem`:

```
.grandchild {
  font-size: 1.5rem; /* 1.5 * 16px = 24px -- hoan toan khong bi ke thua tu cha! */
}
```

Trường Hợp 9: Hiểm Họa Khi Dùng Cùng Một Class .box Với font-size: 1.5em Cho Nhiều Cấp:

Nếu các phần tử con – cháu – chắt đều sử dụng chung một class tái sử dụng (ví dụ `.box`), thì khi gán `font-size` bằng đơn vị `em`, hiện tượng phóng đại kế thừa vẫn xảy ra dữ dội vì `em` phụ thuộc vào `font-size` của phần tử cha trực tiếp:

Ví Dụ 9: Dùng Chung Một Class .box Và font-size: 1.5em Cho Các Cấp Lồng Nhau

```
<div class="box">
  Ong (1)
  <div class="box">
    Bo (2)
    <div class="box">
      Con (3)
      <div class="box">
        Chau (4)
      </div>
    </div>
  </div>
</div>

<style>
  .box {
    font-size: 1.5em;
    background-color: #0074D9;
    color: white;
    padding: 10px;
    margin-top: 10px;
  }
</style>
```

[Bảng] Bảng Phân Tích Phóng Đại Cấp Số Nhân Khi Dùng Cùng Class .box

Phân Cấp	Điểm Tựa Kế Thừa	font-size Tính Ra (px)
.box cấp 1	Không có cha nào ⇒ Kế thừa từ html (1em = 16px)	1.5×16px = **24**px
.box cấp 2	Kế thừa từ cấp 1 (24px)	1.5em $\times$ 24px = **36**px
.box cấp 3	Kế thừa từ cấp 2 (36px)	1.5em $\times$ 36px = **54**px
.box cấp 4	Kế thừa từ cấp 3 (54px)	1.5em $\times$ 54px = **81**px (chữ khổng lồ!)

[Cảnh Báo] Lưu Ý Sống Còn & 2 Cách Phòng Tránh Khi Tái Sử Dụng Class

Khi dùng chung một class như `.box` cho nhiều cấp lồng nhau với đơn vị `em`, hãy luôn ghi nhớ rằng `em` kế thừa từ cha trực tiếp, do đó rất dễ gây ra hiện tượng kích thước chữ bị phóng đại mất kiểm soát.

Hai giải pháp kiến trúc được khuyến nghị:

1. **Dùng `rem` thay cho `em`**: Giúp kích thước luôn bất biến độc lập với cấu trúc lồng nhau:

```
.box {
  font-size: 1.5rem; /* Luon luon co dinh bang 1.5 * 16px = 24px o moi cap! */
}
```

2. **Chỉ định rõ cỡ chữ cho từng cấp bậc**: Tái sử dụng class `.box` cho giao diện chung, nhưng bổ sung modifier class để kiểm soát cỡ chữ:

```
.box.level-1 { font-size: 16px; }
.box.level-2 { font-size: 24px; }
.box.level-3 { font-size: 20px; }
```

Đọc Thêm & Mở Rộng: Góc Chuyên Sâu Về Đơn Vị em (Under The Hood, a11y & Phỏng Vấn)

Để hoàn thiện kiến thức của một kỹ sư Frontend chuyên nghiệp và sẵn sàng cho các buổi phỏng vấn kỹ thuật cấp cao, mục này tổng hợp các nguyên lý vận hành tầng sâu, các bẫy mã nguồn tinh vi và tiêu chuẩn tiếp cận chuẩn quốc tế liên quan đến đơn vị `em`.

1. Góc Phỏng Vấn Tuyển Dụng: Bẫy Kinh Điển line-height: 1.5 vs line-height: 1.5em:

Một trong những câu hỏi phỏng vấn phổ biến nhất nhưng lại khiến rất nhiều lập trình viên mất điểm chính là sự khác biệt giữa việc đặt `line-height` bằng số không đơn vị (**unitless**) và đặt bằng đơn vị `em`.

[Khái Niệm] Bản Chất Toán Học Của Bẫy line-height

- **Khi dùng `line-height: 1.5em`**: Kích thước chiều cao dòng được trình duyệt **tính toán thành pixel cố định ngay tại phần tử cha**, sau đó truyền giá trị pixel cố định này xuống cho tất cả các phần tử con kế thừa.
- **Khi dùng `line-height: 1.5` (không đơn vị)**: Phần tử cha **chỉ truyền tỷ lệ thuần túy** 1.5 xuống cho các phần tử con. Mỗi phần tử con sẽ tự nhân tỷ lệ 1.5 này với chính cỡ chữ riêng của nó.

Mã Nguồn Minh Họa Lỗi Đè Chữ Nghiêm Trọng Với line-height: 1.5em

```
<!-- TRUONG HOP NGUY HIEM: Dung 1.5em -->
<div class="parent-bad">
  Cha (20px, line-height: 1.5em -> tinh ra 30px)
  <h1 class="child-bad">Tieu De Con Co Co Chu 40px</h1>
</div>

<style>
.parent-bad {
  font-size: 20px;
  line-height: 1.5em; /* Cha tinh: 1.5 x 20px = 30px co dinh truyen xuong! */
}
.child-bad {
  font-size: 40px;    /* Co chu lon 40px nhung line-height chi co 30px! */
}
</style>
```

BROWSER PREVIEW **Mô Phỏng Thực Tế: Thảm Họa Chữ Đè Lên Nhau Khi Dùng line-height: 1.5em**

Đoạn văn cha (`font-size: 20px`, `line-height` tính ra cố định: 30px)

DÒNG TIÊU ĐỀ THỨ NHẤT CÓ FONT 40PX
DÒNG THỨ HAI BỊ ĐÈ LÊN DÒNG THỨ NHẤT DO LINE-HEIGHT QUÁ NHỎ (30PX)!

(**Cảnh báo**: Vì line-height chỉ có 30px trong khi cỡ chữ lên tới 40px, các dòng chữ của tiêu đề con sẽ đè bẹp lên nhau!)

[Mẹo] Quy Tắc Vàng Tuyệt Đối Cho line-height

Luôn luôn sử dụng số không đơn vị (unitless) cho thuộc tính `line-height`:

```
/* Chuan muc ky thuat dung: */
body {
  line-height: 1.5; /* Khong co chu 'em' phia sau! */
}
```

Khi đó, tiêu đề con 40px sẽ nhận `line-height` là $1.5 \times 40\text{px} = 60\text{px} \rightarrow$ văn bản giãn cách hoàn hảo, không bao giờ bị đè chữ.

2. Câu Hỏi Phỏng Vấn Senior: Khi Nào Bắt Buộc NÊN Dùng em Thay Vì rem?

Trong kỷ nguyên CSS hiện đại, mặc dù `rem` là đơn vị thống trị cho Typography và Layout toàn trang, nhưng `em` vẫn là **vũ khí không thể thay thế** trong các kịch bản sau:

1. **Tính đóng gói thành phần (Component Encapsulation)**: Khi bạn xây dựng một thành phần UI dùng chung (Design System Component như Button, Badge, Chip, Avatar, Tooltip), thành phần này có thể được gắn vào Sidebar (chữ nhỏ 12px), vào Body (chữ 16px) hoặc vào Hero Banner (chữ lớn 28px).
   - Nếu dùng `rem`: Khoảng đệm `padding` và góc bo `border-radius` sẽ cố định toàn cục, nút bấm ở Hero banner sẽ trông rất mất cân đối (chữ khổng lồ nhưng viền và đệm lại nhỏ xíu).
   - Nếu dùng `em`: Nút bấm tự động co giãn toàn bộ khoảng đệm, khoảng cách icon và góc bo tỷ lệ thuận tuyệt đối theo cỡ chữ của ngữ cảnh chứa nó.
2. **Đồng bộ kích thước biểu tượng đi kèm văn bản (Inline Icons)**: Khi đặt kích thước cho icon SVG nằm cạnh chữ (`width: 1em; height: 1em;`), icon sẽ luôn luôn có kích thước bằng đúng cỡ chữ hiện tại của thẻ cha, không cần viết thêm code điều chỉnh khi thay đổi phông chữ.

3. Tiêu Chuẩn Tiếp Cận Người Dùng (Accessibility – a11y & WCAG 2.1):

Tiêu chuẩn tiếp cận nội dung Web quốc tế (**WCAG 2.1**) tại tiêu chí **1.4.4 (Resize Text)** quy định: Văn bản trên trang web phải có khả năng phóng to lên ít nhất **200%** mà không bị vỡ giao diện hoặc mất nội dung.

[Bảng] So Sánh Khả Năng Đáp Ứng Chuẩn Trợ Năng (a11y) Giữa px Và em / rem

Hành Động Của Người Dùng	Hành Vi Khi Dùng px Cố Định	Hành Vi Khi Dùng em / rem
Thay đổi cỡ chữ mặc định trong Cài đặt trình duyệt (từ 16px lên 24px hoặc 32px)	[Thất Bại] Trang web giữ nguyên cỡ chữ cứng, người thị lực kém không thể đọc được.	[Thành Công Tuyệt Đối] Toàn bộ chữ và thành phần dùng em/rem tự động phóng to đồng bộ theo thị lực người dùng.
Zoom trình duyệt (Phím tắt Ctrl + / -)	Phóng to toàn bộ pixel theo tỷ lệ Viewport.	Phóng to mượt mà, hài hòa với cấu trúc bố cục tự thích ứng.

4. Cơ Chế Bên Trong Cỗ Máy Trình Duyệt (Browser Engine & Under The Hood):

Trong quá trình xử lý mã nguồn của trình duyệt (Blink trên Chrome, Gecko trên Firefox, WebKit trên Safari), đơn vị em trải qua vòng đời tính toán nghiêm ngặt:

[Khái Niệm] Vòng Đời Chuyển Hóa Của Giá Trị em Trong Trình Duyệt

Declared Value (2em) ⟶ Cascaded Value ⟶ Specified Value ⟶ **ComputedValue(32px)**

- **Giai đoạn Computed Style (Tái tính toán kiểu dáng)**: Trình duyệt xây dựng cây **CSSOM**. Tại đây, mọi đơn vị tương đối (em, %) đều được nhân với giá trị phông chữ của cha để quy đổi thành giá trị số thực tuyệt đối (px) gọi là **Computed Value**.
- **Giai đoạn Layout (Reflow)**: Khi bước vào cây hiển thị (**Render Tree**) và tính toán tọa độ khung hình, cỗ máy dựng hình của trình duyệt **hoàn toàn chỉ làm việc với số pixel thực tế**, không còn bất kỳ đơn vị em nào tồn tại trong quá trình vẽ màn hình.

5. Bản Đồ Phân Vai Đơn Vị Chuẩn Doanh Nghiệp (Enterprise Architecture Cheat Sheet):

Để xây dựng một hệ thống giao diện web bền vững, dễ bảo trì và có khả năng mở rộng không giới hạn, các kỹ sư phần mềm chuyên nghiệp luôn tuân theo ma trận phân vai sau:

[Bảng] Ma Trận Phân Vai Các Đơn Vị Đo Lường Trong Dự Án Chuyên Nghiệp

Thành Phần Giao Diện	Đơn Vị Tối Ưu	Lý Do Kỹ Thuật & Trải Nghiệm Người Dùng (UX)
Cỡ chữ toàn trang (h1-h6, p, span)	rem	Giữ cho toàn bộ trang web đồng bộ theo một nguồn chân lý gốc (**Single Source of Truth**) từ thẻ `<html>`.
Bố cục vĩ mô (Grid columns, Flexbox layout, Margin lớn)	rem / % / vw	Giúp khung bố cục ổn định, tránh bị xô lệch khi các thành phần con thay đổi cỡ chữ.
Micro-components (Button, Badge, Chip, Avatar, Tooltip)	em	Cho phép toàn bộ khoảng đệm (`padding`), bo góc (`border-radius`) tự co giãn hài hòa theo kích thước chữ của component.
Biểu tượng đi kèm chữ (Inline SVG Icons)	em	Icon tự động bằng đúng chiều cao chữ (`width: 1em; height: 1em;`), luôn thẳng hàng tuyệt đối.
Khoảng cách dòng (line-height)	Unitless (1.5)	Ngăn ngừa triệt để bẫy đè chữ khi phần tử con kế thừa từ cha.
Đường viền mỏng (border)	px (1px, 2px)	Đảm bảo nét vẽ siêu mảnh, không bị mờ nhòe do làm tròn số thập phân trên màn hình độ phân giải cao.

1.5.5 Chuyên Đề Đơn Vị `rem` (rem Masterclass Trong CSS)

1. Bản Chất Đơn Vị rem & 3 Đặc Điểm Cốt Lõi:

`rem` là viết tắt của thuật ngữ **"Root EM"**. Đây là đơn vị đo lường tương đối được tính toán dựa trên kích thước phông chữ (`font-size`) của **phần tử gốc duy nhất của trang web là thẻ `<html>`**. Khác biệt hoàn toàn với `em`, `rem` không bị ảnh hưởng bởi thẻ cha gần nhất, mà luôn luôn tính từ thẻ gốc `html`.

[Khái Niệm] 3 Đặc Điểm Nền Tảng Của Đơn Vị rem

1. **Tương đối với phần tử gốc (`<html>`)**: Đây là điểm khác biệt lớn nhất so với đơn vị `em`. Trong khi `em` tương đối với `font-size` của phần tử cha trực tiếp, `rem` luôn tương đối với `font-size` của thẻ `<html>`. Điều này giúp triệt tiêu hoàn toàn "hiệu ứng tầng" (**Cascading / Compounding Effect**) mà `em` có thể gây ra khi các phần tử lồng nhau.
2. **Kiểm soát kích thước đồng bộ**: Khi bạn thay đổi `font-size` của thẻ `<html>`, tất cả các phần tử sử dụng đơn vị `rem` trên trang sẽ tự động điều chỉnh kích thước theo tỷ lệ. Điều này cực kỳ hữu ích cho việc thiết kế giao diện có khả năng co giãn tỷ lệ thống nhất (**Scalable Design**) và dễ dàng thay đổi kích thước toàn trang.
3. **Thân thiện với Responsive và Accessibility (Trợ năng)**:
   - **Responsive**: Bạn có thể sử dụng `rem` để định nghĩa kích thước các thành phần (như `padding`, `margin`, `width`, `height`, `font-size`) sao cho chúng tự động thay đổi theo kích thước màn hình bằng cách điều chỉnh `font-size` của `<html>` trong các Media Queries.
   - **Accessibility**: Người dùng có thể dễ dàng điều chỉnh kích thước văn bản của trang web thông qua cài đặt trình duyệt (ví dụ: phóng to / thu nhỏ văn bản). Khi sử dụng `rem`, văn bản và các thành phần khác sẽ tự động thích ứng với cài đặt của người dùng, cải thiện trải nghiệm vượt bậc cho những người có thị lực kém.

2. Cơ Chế Hoạt Động & Công Thức Quy Đổi Toán Học:

Đơn vị `rem` hoạt động dựa trên nguyên tắc tương đối với kích thước phông chữ của phần tử gốc duy nhất của tài liệu là thẻ `<html>`.

[Khái Niệm] Quy Trình 3 Bước Vận Hành Của Đơn Vị rem

1. **Bước 1 – Trình duyệt đọc `font-size` của thẻ `<html>`**: Mặc định là 16px nếu không bị ghi đè trong CSS.
2. **Bước 2 – Xác lập quy chuẩn cơ sở**: Mỗi 1rem = giá trị font-size của `<html>`.
3. **Bước 3 – Áp dụng đồng bộ**: Mọi phần tử sử dụng đơn vị `rem` trên toàn trang sẽ luôn tính toán dựa trên thẻ gốc `<html>`, **hoàn toàn KHÔNG bị ảnh hưởng bởi bất kỳ thẻ cha trung gian nào**.

a. Điểm khởi đầu – Kích thước phông chữ của thẻ gốc `<html>`: Mọi tính toán với `rem` đều bắt đầu từ việc xác định giá trị `font-size` của thẻ `<html>`:

- **Mặc định trình duyệt**: Nếu bạn không thiết lập `font-size` cho thẻ `<html>`, hầu hết mọi trình duyệt (Chrome, Firefox, Edge, Safari,...) sẽ mặc định dùng 16px làm kích thước cơ sở. Khi đó: **1**rem = **16**px.
- **Thiết lập thủ công**: Bạn có thể tự định nghĩa `font-size` cho thẻ `<html>` trong file CSS để kiểm soát kích thước cơ sở:

Thiết Lập Mặc Định: font-size Cơ Sở Cho Thẻ html

```
1 html {
2   font-size: 16px; /* 1rem = 16px */
3 }
```

Hoặc để thuận tiện trong việc nhẩm toán, nhiều lập trình viên áp dụng tỷ lệ 62.5%:

Kỹ Thuật 62.5%: Chuyển Đổi Hệ Quy Chiếu Về Cơ Số 10 Dễ Nhẩm

```
1 html {
2   font-size: 62.5%; /* Tuong duong 10px neu mac dinh trinh duyet la 16px */
3   /* Cach tinh: 16px * 0.625 = 10px */
4   /* Khi do: 1rem = 10px. Tinh toan tro nen sieu de dang (vi du: 1.5rem = 15px)
      */
5 }
```

Dù bạn thiết lập giá trị nào, `font-size` của thẻ `<html>` sẽ trở thành **"hệ số nhân"** cho tất cả các giá trị `rem` xuất hiện trên toàn bộ trang web.

b. Tính toán giá trị thực tế (Pixel) trên đa dạng thuộc tính: Khi bạn sử dụng `rem` cho **bất kỳ thuộc tính CSS nào** (như `font-size`, `padding`, `margin`, `width`, `height`, `border-radius`,...), trình duyệt sẽ thực hiện phép nhân đơn giản để chuyển đổi giá trị `rem` thành giá trị pixel thực tế mà nó render lên màn hình.

[Khái Niệm] Công Thức Chuyển Đổi Đơn Vị rem Sang Pixel Thực Tế

$$\text{Giá Trị Thực Tế (px)} = \text{Giá Trị rem} \times \text{font-size của thẻ gốc } \texttt{<html>}$$

Theo chuẩn mặc định:

$$1\text{rem} = 16\text{px}, \quad 2\text{rem} = 32\text{px}, \quad 0.5\text{rem} = 8\text{px}$$

Xét ví dụ minh họa tính toán trên nhiều thuộc tính thuộc Box Model và Typography:

Khai Báo Nhiều Thuộc Tính Box Model & Typography Bằng Đơn Vị rem

```
html {
  font-size: 16px;    /* Gia tri goc tham chieu */
}

h1 {
  font-size: 2.5rem;  /* Co chu tieu de */
  margin-top: 1.25rem;/* Khoang cach le tren */
}

p {
  font-size: 1rem;    /* Co chu doan van */
  padding: 0.5rem;    /* Khoang dem noi bo */
}

.box {
  width: 10rem;       /* Chieu rong */
  height: 5rem;       /* Chieu cao */
}
```

Cách trình duyệt tính toán giá trị pixel thực tế kết xuất ra màn hình:

- **Phần tử `<h1>`**:
  - `font-size: 2.5rem` $\rightarrow 2.5 \times 16\text{px} = \mathbf{40}\text{px}$.
  - `margin-top: 1.25rem` $\rightarrow 1.25 \times 16\text{px} = \mathbf{20}\text{px}$.
- **Phần tử `<p>`**:
  - `font-size: 1rem` $\rightarrow 1 \times 16\text{px} = \mathbf{16}\text{px}$.
  - `padding: 0.5rem` $\rightarrow 0.5 \times 16\text{px} = \mathbf{8}\text{px}$.
- **Phần tử `.box`**:
  - `width: 10rem` $\rightarrow 10 \times 16\text{px} = \mathbf{160}\text{px}$.
  - `height: 5rem` $\rightarrow 5 \times 16\text{px} = \mathbf{80}\text{px}$.

[Bảng] Bảng Tra Cứu Quy Đổi Nhanh Từ Pixel Sang rem (Chuẩn Gốc 16px)

Mục Đích Dùng	Pixel (px)	Giá Trị rem	Giải Thích Quy Đổi
Chú thích nhỏ (Small Text)	12px	0.75rem	$12/16 = 0.75$rem.
Chữ phụ (Secondary Text)	14px	0.875rem	$14/16 = 0.875$rem.
Văn bản chuẩn (Body Text)	16px	1rem	$16/16 = 1$rem (Chuẩn mặc định trình duyệt).
Tiêu đề phụ (Subheading)	20px	1.25rem	$20/16 = 1.25$rem.
Tiêu đề H2 (Heading 2)	24px	1.5rem	$24/16 = 1.5$rem.
Tiêu đề H1 (Heading 1)	32px	2rem	$32/16 = 2$rem.
Tiêu đề Hero Banner	48px	3rem	$48/16 = 3$rem.

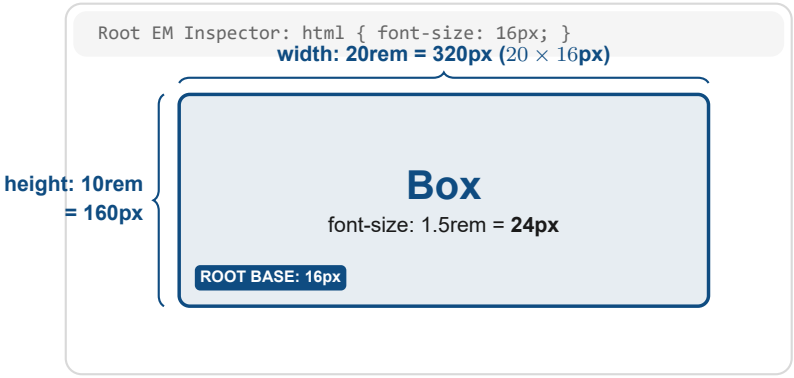
3. Các Ví Dụ Trực Quan Minh Họa Đơn Vị rem:

Ví Dụ 3.1: Sử Dụng rem Cho Kích Thước Bố Cục Và Phông Chữ

```
<div class="box">Box</div>

<style>
html {
  font-size: 16px;
}
.box {
  width: 20rem;       /* = 20 x 16px = 320px */
  height: 10rem;      /* = 10 x 16px = 160px */
  font-size: 1.5rem;  /* = 1.5 x 16px = 24px */
  background: lightblue;
}
</style>
```

BROWSER PREVIEW Mô Phỏng Trực Quan: Khối .box Thiết Lập Toàn Bộ Bằng rem



(→ Kết quả: Chiều rộng 320px, chiều cao 160px và cỡ chữ 24px đều tính chuẩn mực theo gốc thẻ `html`).

Ví Dụ 3.2: Thay Đổi Cỡ Chữ Root Thẻ html Ảnh Hưởng Đồng Bộ Toàn Bộ Trang

```
<style>
html {
  font-size: 10px;    /* Thay doi co chu goc thanh 10px */
}
.box {
  font-size: 2rem;    /* = 2 x 10px = 20px */
}
</style>
```

[Ghi Chú] Nguyên Lý Vận Hành Đồng Bộ Của rem

Khi bạn thay đổi `font-size` của thẻ gốc `html`, toàn bộ các kích thước xây dựng dựa trên đơn vị `rem` trên khắp trang web sẽ tự động co giãn theo tỷ lệ mới một cách đồng nhất, giúp việc scale giao diện trở nên vô cùng nhẹ nhàng.

Ví Dụ 3.3: Sự "Ma Thuật" Của rem Trong Thiết Kế Đáp Ứng (Responsive Design)

```
/* 1. Gia tri mac dinh danh cho man hinh Desktop lon (> 768px) */
html {
  font-size: 16px;    /* 1rem = 16px */
}
h1 {
  font-size: 2.5rem;  /* 2.5 x 16px = 40px */
}
p {
  font-size: 1rem;    /* 1 x 16px = 16px */
}

/* 2. Khi man hinh Tablet co chieu rong toi da 768px */
@media (max-width: 768px) {
  html {
    font-size: 14px;  /* Giam font-size goc xuong 14px */
  }
}

/* 3. Khi man hinh Mobile co chieu rong toi da 480px */
@media (max-width: 480px) {
  html {
    font-size: 12px;  /* Giam font-size goc xuong 12px */
  }
}
```

[Bảng] Ma Trận Co Giãn Kích Thước Tự Động Theo Thiết Bị Nhờ rem

Ngưỡng Màn Hình	font-size Thẻ html	Tiêu Đề h1 (2.5rem)	Đoạn Văn p (1rem)
Màn hình lớn (> 768px)	16px (Chuẩn)	2.5×16px $=$ **40px**	1×16px $=$ **16**px
Màn hình vừa (481px $- 768$px)	14px	2.5×14px $=$ **35px**	1×14px $=$ **14**px
Màn hình nhỏ (≤ 480px)	12px	2.5×12px $=$ **30px**	1×12px $=$ **12**px

[Mẹo] Lợi Ích Vượt Trội Của rem Trong Kiến Trúc Giao Diện

Bạn không cần phải viết lại CSS cho từng phần tử (`h1`, `p`, `.box`,...) trong mỗi Media Query. Chỉ cần điều chỉnh duy nhất `font-size` của `<html>`, toàn bộ hệ thống giao diện từ phông chữ đến khoảng cách bố cục sẽ tự động co giãn đồng bộ theo tỷ lệ hoàn hảo. Điều này giúp mã nguồn tinh gọn, sạch sẽ, dễ bảo trì và giảm thiểu tối đa sai sót.

4. Ma Trận So Sánh Toàn Diện: rem vs em:

Dưới đây là bảng tổng hợp đối chiếu sự khác biệt bản chất giữa hai đơn vị tương đối quan trọng nhất:

[Bảng] Bảng So Sánh Nhanh Đặc Điểm Vận Hành: rem vs em

Đặc Điểm So Sánh	Đơn Vị em	Đơn Vị rem
Phụ thuộc vào	Phần tử cha gần nhất (với font-size) hoặc chính nó (với box model).	**Phần tử gốc duy nhất thẻ** `<html>`.
Dễ dự đoán	[Không] Khó dự đoán nếu các cấp cha lồng nhau cũng dùng em.	[Có] Rất dễ dự đoán vì luôn bám theo chuẩn thẻ `<html>`.
Trường hợp thường dùng	Dùng trong padding, khoảng cách (spacing) nội bộ của component.	Dùng trong thiết lập bố cục (**Layout**) và hệ thống cỡ chữ (**Typography**) toàn trang.

[Bảng] Bảng So Sánh Đối Chiếu Chi Tiết Giữa em Và rem

Tiêu Chí So Sánh	Đơn Vị em	Đơn Vị rem
Điểm tham chiếu	Phần tử cha gần nhất (với `font-size`) hoặc chính nó (với padding/margin).	Thẻ gốc `<html>` duy nhất trên toàn bộ trang.
Độ phức tạp kiểm soát	**[Phức tạp]** Dễ bị nhân dồn kích thước khi lồng nhau nhiều cấp.	**[Rất dễ]** Đồng nhất, dự đoán chính xác kích thước ở mọi nơi trong DOM.
Khả năng mở rộng (Scaling)	Cục bộ theo từng thành phần (Component-level).	Toàn cục trên quy mô cả trang web (Global-level).
Trường hợp tối ưu nhất	Tạo khoảng đệm (`padding`), bo góc (`border-radius`) cho nút bấm co giãn.	Định cỡ phông chữ (`font-size`), khoảng cách bố cục (`margin`, `gap`) chuẩn Responsive.

Thực nghiệm đối chứng: Hành vi của em vs rem trên cùng một cấu trúc DOM lồng nhau: Để hiểu sâu sắc sự khác biệt cốt lõi, hãy quan sát cùng một cây DOM lồng nhau 3 cấp:

Cấu Trúc HTML Cây DOM Lồng Nhau 3 Cấp: .parent > .child > .grandchild

```
<div class="parent">
  Parent Text
  <div class="child">
    Child Text
    <div class="grandchild">
      Grandchild Text
    </div>
  </div>
</div>
```

Thử Nghiệm 1: Định Cỡ Bằng Đơn Vị em (Gây Hiệu Ứng Tầng Compounding)

```
html {
  font-size: 16px;
}
.parent {
  font-size: 1em;   /* 1 x 16px = 16px */
}
.child {
  font-size: 1.2em; /* 1.2 x font-size cua .parent (1.2 x 16px = 19.2px) */
}
.grandchild {
  font-size: 1.2em; /* 1.2 x font-size cua .child (1.2 x 19.2px = 23.04px) */
}
```

[Cảnh Báo] Hệ Quả Của Hiệu Ứng Tầng (Compounding Effect) Khi Dùng em

Với `em`, kích thước phông chữ của `.grandchild` bị ảnh hưởng bởi `.child`, và `.child` lại bị ảnh hưởng bởi `.parent`. Kích thước bị nhân dồn lên thành 23.04px, trở nên rất khó kiểm soát khi cấu trúc DOM lồng nhau phức tạp.

Thử Nghiệm 2: Định Cỡ Bằng Đơn Vị rem (Hoàn Toàn Độc Lập Khỏi Thẻ Cha)

```
html {
  font-size: 16px;
}
.parent {
  font-size: 1rem;   /* 1 x 16px = 16px */
}
.child {
  font-size: 1.2rem; /* 1.2 x 16px = 19.2px */
}
.grandchild {
  font-size: 1.4rem; /* 1.4 x 16px = 22.4px */
}
```

[Khái Niệm] Điểm Tựa Độc Lập Tuyệt Đối Của rem

Với `rem`, kích thước của `.child` (19.2px) và `.grandchild` (22.4px) **hoàn toàn độc lập** với phần tử cha của chúng. Mọi phép tính đều neo chặt vào mỏ neo duy nhất là thẻ `<html>`, giúp kích thước luôn dễ dự đoán và ổn định tuyệt đối trong mọi tình huống.

5. Toàn Bộ Những Lưu Ý Sống Còn Khi Sử Dụng Đơn Vị rem (7 Nguyên Tắc Vàng):

Đơn vị `rem` là mỏ neo trung tâm của hệ thống Typography và Spacing trong CSS hiện đại. Để khai thác tối đa sức mạnh của `rem` mà không gặp phải các lỗi giao diện tiềm ẩn, bạn cần nắm vững 7 nguyên tắc thiết kế then chốt dưới đây:

1. **Thiết lập `font-size` cho thẻ `<html>` một cách cẩn thận**:

   Đây là bước quan trọng nhất vì nó định nghĩa giá trị cơ sở (**Base Value / Single Source of Truth**) cho tất cả các đơn vị `rem` trên toàn bộ trang web của bạn:

   - **Sử dụng giá trị mặc định của trình duyệt (16px) hoặc** 100%: Nếu bạn không đặt `font-size` cho `<html>`, trình duyệt sẽ sử dụng giá trị mặc định của người dùng (thường là 16px). Điều này cho phép người dùng có cài đặt riêng về kích thước chữ. Nếu bạn muốn giữ nguyên điều này, bạn có thể thiết lập rõ ràng `font-size: 100%;` cho `html`:

     Khai Báo font-size 100% Giữ Nguyên Cài Đặt Người Dùng

     ```
     1 html {
     2   font-size: 100%; /* Giu nguyen cai dat mac dinh cua trinh duyet, thuong
           la 16px */
     3 }
     ```

   - **Thiết lập `font-size: 62.5%;` để dễ tính toán**: Nhiều nhà phát triển ưa chuộng thiết lập `font-size: 62.5%;` cho `<html>`:

     Kỹ Thuật 62.5% Cho Thẻ html Giúp Quy Đổi Nhẩm Dễ Dàng

     ```
     1 html {
     2   font-size: 62.5%; /* Tuong duong 10px neu font-size mac dinh cua trinh
           duyet la 16px */
     3 }
     ```

     Lý do: $16\text{px} \times 0.625 = 10\text{px}$. Khi đó, 1rem sẽ tương đương 10px, giúp việc chuyển đổi từ pixel sang `rem` trở nên đơn giản hơn rất nhiều (ví dụ: $24\text{px} = 2.4\text{rem}$, $18\text{px} = 1.8\text{rem}$). Tuy nhiên, cần lưu ý rằng cách này có thể "ghi đè" lên cài đặt mặc định của người dùng, nên hãy cân nhắc kỹ lưỡng.

   - **Không đặt `font-size` quá nhỏ hoặc quá lớn**: Đặt một giá trị `font-size` hợp lý cho `html` để đảm bảo trải nghiệm cơ bản tốt nhất cho người dùng trước khi áp dụng các Media Queries.

2. **Sử dụng `rem` cho các thuộc tính kích thước**:

   Đơn vị `rem` không chỉ dành riêng cho `font-size`. Nó cực kỳ hữu ích cho các thuộc tính liên quan đến kích thước và khoảng cách trong cấu trúc Box Model:

   - **Ưu tiên cho `font-size`, `padding`, `margin`, `line-height`**: Đây là những thuộc tính lý tưởng để sử dụng `rem` vì chúng giúp duy trì tỷ lệ tổng thể của giao diện khi kích thước chữ gốc thay đổi.
   - **Áp dụng cho `width`, `height`, `min-width`, `max-width`, `border-radius`, v.v.**: Nếu bạn muốn các thành phần này tự động co giãn theo tỷ lệ font chữ cơ bản của trang, hãy sử dụng `rem`. Ví dụ: một

nút bấm (`<button>`) có `padding` và `border-radius` được định nghĩa bằng `rem` sẽ tự động co giãn hợp lý khi kích thước màn hình thay đổi.

3. **Cẩn trọng với việc kết hợp `rem` và `px`:**

   Mặc dù `rem` rất mạnh mẽ, đôi khi bạn vẫn cần sử dụng `px` cho một số trường hợp cụ thể:

   - **`px` cho các thuộc tính yêu cầu độ chính xác cao:**
     - **`border-width`**: Thường nên dùng `px` để đảm bảo độ dày của đường viền luôn nhất quán, không bị "biến mất" khi kích thước màn hình quá nhỏ.
     - **`box-shadow`**: Dùng `px` để kiểm soát hiệu ứng đổ bóng chính xác tuyệt đối.
     - **Các icon có kích thước cố định**: Nếu bạn sử dụng các icon có kích thước pixel cụ thể và không muốn chúng co giãn, `px` là lựa chọn tốt nhất.
   - **Kết hợp linh hoạt**: Bạn có thể kết hợp cả `rem` và `px` trong cùng một phần tử. Ví dụ, một phần tử có `padding: 1rem; border: 1px solid black;`.

4. **Tránh "Hiệu ứng tầng" (Compounding Effect) của `em` khi dùng `rem`:**

   Điểm mạnh vượt trội của `rem` là nó loại bỏ hoàn toàn "hiệu ứng tầng" (**Compounding Effect**) mà `em` có thể gây ra trong cấu trúc cây DOM lồng nhau.

   - **Không lạm dụng `em` trong các phần tử lồng nhau**: Nếu bạn muốn một kích thước nhất quán và dễ kiểm soát, hãy ưu tiên `rem`. Chỉ sử dụng `em` khi bạn thực sự muốn kích thước của một phần tử phụ thuộc vào kích thước font của chính nó hoặc của phần tử cha trực tiếp (ví dụ: thuộc tính `line-height` thường dùng số không đơn vị hoặc `em` để tỷ lệ với `font-size` của văn bản đó).

5. **Kiểm tra trên nhiều trình duyệt và thiết bị:**

   Mặc dù `rem` được hỗ trợ rộng rãi 100% trên tất cả trình duyệt hiện đại, việc kiểm tra luôn là yêu cầu bắt buộc:

   - **Responsive testing**: Thử nghiệm giao diện của bạn trên các kích thước màn hình khác nhau (sử dụng Chrome DevTools Device Mode hoặc các thiết bị thực tế) để đảm bảo mọi thứ hiển thị như mong đợi khi `font-size` của `<html>` thay đổi.
   - **Accessibility testing**: Kiểm tra bằng cách phóng to/thu nhỏ văn bản trong cài đặt của trình duyệt (Browser Zoom & Text Zoom) để xem các phần tử có co giãn một cách phù hợp và trang web vẫn dễ đọc hay không.

6. **Sử dụng `rem` trong các Media Queries:**

   Đây là kỹ thuật cốt lõi và thanh lịch nhất để tạo ra thiết kế Responsive với `rem`:

   - **Điều chỉnh `font-size` của thẻ gốc `<html>`**: Thay vì phải viết lại tất cả các giá trị `rem` cho từng breakpoint của mỗi phần tử riêng biệt, bạn chỉ cần thay đổi giá trị `font-size` cơ sở của thẻ `<html>`:

Scale Toàn Bộ Typography & Spacing Bằng Media Query Trên html

```
/* Base font size cho Desktop */
html {
  font-size: 16px;
}

/* Dieu chinh cho May tinh bang (Tablets) */
@media (max-width: 768px) {
  html {
    font-size: 14px;
  }
}

/* Dieu chinh cho Dien thoai di dong (Mobile) */
@media (max-width: 480px) {
  html {
    font-size: 12px;
  }
}
```

Cách này giúp bạn kiểm soát toàn bộ typography và spacing của toàn trang web một cách tập trung, nhanh chóng và hiệu quả vượt bậc.

7. **Cân nhắc việc sử dụng `clamp()` cho typography động (Fluid Typography)**:
   Đối với các dự án lớn hơn hoặc muốn kiểm soát tốt hơn sự co giãn mượt mà của `font-size`, bạn có thể kết hợp `rem` với hàm `clamp()` trong CSS:
   - **Cú pháp `clamp(min, preferred, max)`**: Cho phép bạn đặt một kích thước tối thiểu, một kích thước ưu tiên (sử dụng kết hợp `vw` hoặc `rem`) và một kích thước tối đa cho font chữ:

Fluid Typography Mượt Mà Không Cần Media Query Bằng clamp()

```
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
  /* Font size se nam trong khoang 2rem va 4rem,
     uu tien 5% cua viewport width (vw) */
}
```

Điều này mang lại sự linh hoạt cao hơn trong việc điều chỉnh kích thước văn bản một cách mượt mà vô cấp trên các kích thước màn hình khác nhau mà không bị ngắt quãng giật cục.